Bachelor of Science in Computer Science
May 2024



A Comparative Analysis of RIPPER and CN2 Algorithms in Personal Loan Credit Scoring

Naga Pujitha Pitta
Joshna Sindhuja Pitta

Faculty of Engineering, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This thesis was submitted to the Faculty of Computer Science Engineering at Blekinge Institute of Technology to fulfill the Bachelor of Science degree requirements in computer science. The study for this thesis extended 20 weeks of full-time academic involvement.

The authors confirmed their authorship of this thesis and the use of just the sources specified in the bibliography and acknowledged them as references. Also, they state that this thesis has not been presented to any other academic institution in search of a degree.

Contact Information:
Author(s):
Naga Pujitha Pitta
E-mail: napi23@student.bth.se

Joshna Sindhuja Pitta
E-mail: jopi23@student.bth.se

University advisor:
Dr. Prashant Goswami
Department of Computer Science and Engineering

Dept. of Mathematics and Natural Sciences
Blekinge Institute of Technology
SE–371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Background: Credit scoring is a mechanism used by lenders and financial organizations to determine the creditworthiness of individuals or businesses requesting credit or loans. It involves assessing various indicators to determine the likelihood of a borrower repaying their loans on time. The primary goal of credit scoring is to evaluate the risk involved in lending money to a particular borrower. Additionally, rule-based machine learning algorithms are employed to determine whether an individual is eligible to receive a loan.

Objectives: The thesis compared the RIPPER and CN2 algorithms to determine their usefulness in predicting personal loan credit scores. The primary objective was to assess model performance using accuracy, precision, recall, and F1 score metrics. The study aimed to determine which of these two rule-based machine learning algorithms gave more accurate and dependable predictions in the context of personal loan credit scores.

Method: The thesis used an experimentation methodology to achieve its objectives and answer the research questions. The Credit Score Classification dataset from Kaggle is chosen which is extremely helpful since it provides critical data to determine creditworthiness, which is an important part of the banking business. Using this method, a comparison was made between two rule-based machine learning (ML) algorithms, RIPPER and CN2. The experimentation step involved rigorous training, validation, and testing of these algorithms on the chosen dataset. Performance evaluation was then carried out, concentrating on key measures such as accuracy, precision, recall, and F1 score. This allowed for a complete comparison of the algorithm's performance in predicting credit scores for personal loans. Hyperparameter refining optimized model performance through extensive testing with different parameter values. Model validation was carried out using k-fold cross-validation. Finally, the study aimed to find the superior algorithm based on the utilized performance metrics.

Results: According to the experiment results, CN2 outperformed the two chosen machine learning algorithms, with an accuracy rate of 47.51% against RIPPER's 38.24%. The precision analysis found that CN2 has a somewhat greater precision of 42%, compared to RIPPER's value of 34%. However, RIPPER had a greater recall of 58% than CN2's 43%. Despite this, CN2 achieved a higher F1 score of 42%, above RIPPER's score of 38%, indicating that it performed better overall on the testing dataset.

Conclusions: CN2 outperformed RIPPER in accuracy, precision, and F1 score, indicating its ability to make accurate predictions when identifying creditworthy loan applicants. While RIPPER had a slightly higher recall, suggesting its effectiveness in identifying positive cases, CN2 achieved a better balance between precision and recall. CN2's more balanced and consistent performance across all measures makes it the preferred option for personal loan credit scoring applications.

Keywords: Machine Learning, RIPPER, CN2, Rule-Based Models, Credit Scoring.

Acknowledgments

We want to express our heartfelt gratitude to Dr. Prashant Goswami, who served as our supervisor and provided constant support and advice during our thesis. His insightful remarks and guidance have helped shape our thinking and improve our abilities. His constructive feedback has encouraged us to further research and inspired us to improve our skills. His commitment to our academic development has been invaluable, and we are thankful for his guidance throughout this process.

Contents

Abstract **3**

Acknowledgments **5**

1 Introduction **13**
1.1 Ethical, societal and sustainability aspects 16
1.1.1 Ethical considerations 16
1.1.2 Societal considerations 16
1.2 Aim and objectives 17
1.2.1 Aim 17
1.2.2 Objectives 17
1.3 Research questions 17
1.4 Scope 18
1.5 Outline 18

2 Background **19**
2.1 Rule-based classifiers 19
2.2 Rule induction 21
2.3 Sequential covering algorithms 21
2.4 RIPPER algorithm 22
2.4.1 Learn-one-rule 22
2.4.2 Rule pruning 22
2.5 CN2 algorithm 22
2.5.1 Learn-one-rule 22
2.5.2 Rule pruning 23
2.6 Data preprocessing and model evaluation techniques 23
2.6.1 Label encoding 23
2.6.2 Min-Max scaler 23
2.6.3 Train-test-split 24
2.6.4 Validation 24
2.6.5 K-Fold cross validation 24
2.6.6 Hyperparameter tuning 24
2.7 Performance Metrics 25
2.7.1 Confusion matrix 25

3 Related Work **27**
3.1 Research Gap 30

4 Method **31**
4.1 Working environment . . . 33
4.1.1 Libraries . . . 33
4.2 Data collection . . . 34
4.3 Data preprocessing . . . 36
4.3.1 Handling missing & null values . . . 36
4.3.2 Encoding categorical variables . . . 36
4.3.3 Data normalization . . . 37
4.3.4 Removing unwanted features . . . 37
4.3.5 Outlier detection . . . 38
4.4 Proportion probabilities . . . 40
4.5 Data splitting . . . 40
4.6 Model training . . . 41
4.6.1 RIPPER . . . 41
4.6.2 CN2 . . . 41
4.7 Hyperparameter tuning . . . 42
4.8 Model evaluation . . . 43
4.9 Model validation . . . 43

5 Results and Analysis **45**
5.1 Rules generated by RIPPER and CN2 . . . 45
5.2 Results of the experiment . . . 47
5.2.1 Accuracy . . . 47
5.2.2 Precision . . . 48
5.2.3 Recall . . . 48
5.2.4 F1 score . . . 48
5.3 Analysis . . . 49

6 Discussion **51**
6.1 Reflections . . . 54

7 Conclusions and Future Work **57**
7.1 Conclusion . . . 57
7.2 Future Work . . . 58

References **59**

A Supplemental Information **65**

List of Figures

4.1 Experimentation procedure 32
4.2 Overview of the dataset 36
4.3 The image shows the conversion of category data to numerical representation 37
4.4 The picture shows how the min-max scaler changes numerical features to fit within a specific range of values 37
4.5 Removing unwanted features from the dataset 38
4.6 Count of the outliers detected 38
4.7 Outliers identified in the dataset 39
4.8 The picture displays the outliers both before and after their detection. 39
4.9 The figure represents the balanced proportion probability for the selected data. 40
4.10 Train-test split function 41
4.11 The diagram for K-fold cross-validation. 44

5.1 Rules generated by RIPPER algorithm 46
5.2 Rules generated by CN2 algorithm 47

6.1 Comparison based on accuracy 52
6.2 Comparison based on precision 52
6.3 Comparison based on recall 53
6.4 Comparison based on F1 score 54

List of Tables

1.1 Motivation for choosing the RIPPER and CN2 algorithms for credit scoring 15

4.1 Description of attributes in the dataset 35

5.1 Accuracy of the test data 47
5.2 Precision of the test data 48
5.3 Recall of the test data 48
5.4 F1 score of the test data 49

Chapter 1

Introduction

Credit scoring is a frequently used mathematical and statistical tool for determining the risk of individuals requesting credit. These approaches evaluate risk by considering the applicant's credentials and past data [39]. Credit scoring is the process of assessing a person's or company's creditworthiness by statistical methods when they request credit. To assist lenders in making well-informed decisions on loan approvals and interest rates, it considers variables such as credit history, income, and employment status. Credit scoring helps lenders manage risk and keep a healthy credit profile by calculating the likelihood of default. It's an essential tool for evaluating credit risk, expediting the loan application process, and increasing productivity [27].

Credit analysts at banks and mail-order corporations decide whether to make loans or send things based on predetermined criteria. These criteria were reduced and applied by non-experts to aid in credit decisions, establishing an early example of expert systems. When credit cards gained popularity in the late 1960s, banks and other lenders began to use credit rating. This improved prediction accuracy and reduced the number of persons who failed to repay loans by at least half. Credit scoring became more popular in the 1970s, increasing the number of experts in the field. By the 1980s, credit scoring had become standard for personal loans, mortgages, small business loans, and other financings. Scorecards were first used in credit scoring in the 1990s [25].

Credit scoring is divided into two major components: "credit" and "scoring." To begin, the term "credit" refers to the concept of "buy now, pay later," which derives from the Latin word "credo," which means "I believe" or "I trust in." Second, "scoring" refers to using a numerical method to rate situations according to perceived quality, allowing for objective and consistent decision-making. A credit score is like assigning a grade to someone's ability to manage money properly. This grade is based on statistical models that examine several aspects of a person, such as payment history, income, and debt. The grade, or score, can be expressed as a number or a letter, and it assists lenders in determining whether and how much to lend to someone. These models additionally predict the chance of a borrower failing to repay a loan, a process known as default. Credit scoring is used for various purposes, including business and real estate transactions. It enables lenders to make fair and consistent decisions about who to lend money to, how much to lend, and how to manage their lending operations such that both the borrower and the lender benefit financially [3].

The study focuses on examining two distinct rule-based algorithms [36], RIPPER (Repeated Incremental Pruning to Produce Error Reduction) and CN2 (ClassiNet 2), in the context of personal loan credit scoring. The major goal is to compare these algorithms. The RIPPER and CN2 algorithms are both used for rule induction in the context of machine learning [21]. The RIPPER algorithm is a rule-based classifier that uses a two-phase approach. In the first phase, it constructs an initial set of rules using a separate-and-conquer strategy, where each rule covers as many training examples as possible. In the second phase, the algorithm prunes the rule set by removing rules that do not contribute to the overall accuracy of the classifier. This iterative process of rule construction and pruning continues until no further improvement in accuracy is achieved [19].

On the other hand, the CN2 algorithm is also a rule-based classifier that follows a similar two-phase approach. It starts by constructing an initial rule set using a separate-and-conquer strategy, and then it iteratively adds rules to the set, each time considering the most informative attribute. The algorithm uses a statistical measure to determine the best attribute to add at each iteration. Similar to RIPPER, CN2 also prunes the rule set to improve the overall accuracy of the classifier [22].

The motivation for using the RIPPER and CN2 algorithms for credit scoring applications begins from their insufficient utilization, but their inherent advantages. RIPPER was chosen for its simplicity and accessibility, making it suitable for study into personal loan scores, where interpretability is essential. The ability to handle noisy datasets increased interest in its application. However, CN2's ability to generate clear and accurate rules has only been heavily investigated in credit scoring, which requires a comparison with RIPPER. This study aims to fill a significant gap in the literature, as existing studies primarily focus on well-researched algorithms such as the Naive Bayesian Model, Logistic Regression Analysis, and Random Forest, leaving a limited amount of research on RIPPER and CN2. This study aims to examine various rule-based platforms to provide novel ideas and contribute to improving personal loan score algorithms.

This thesis stands as a different research attempt within the field of personal loan credit scoring. While several algorithms, like Naive Bayesian Model, Logistic Regression Analysis, Random Forest, Decision Tree, and K-Nearest Neighbor Classifier have been widely investigated [44], there is a significant gap in research focusing on the RIPPER and CN2 algorithms. This work seeks to address this gap by carefully studying these rule-based models in the context of personal loan credit scoring.

Aspect	**RIPPER**	**CN2**
Transparency	Creates simple rules that help loan officers and borrowers make clear decisions.	Provides interpretable rules, while they may be more complex, which aids in understanding decision criteria.
Efficiency	vast datasets are processed rapidly, making them useful for efficiently managing vast amounts of credit data.	It is efficient, but may be slower with large datasets; however appropriate for moderate-sized credit data.
Handling Noisy Data	Moderately robust; minimizes the influence of noisy data, which is typical in financial datasets.	Highly robust; successfully manages noisy and poor credit data, increasing reliability.
Rule Complexity	Create simple, clear guidelines for easier communication of credit decisions.	Can create more extensive and complicated rules, resulting in a more detailed understanding of creditworthiness.
Flexibility in Modeling	Focuses on creating precise, straightforward rules for well-defined score tasks.	Provides flexibility through the use of numerous rule types that can be applied to different areas of credit scoring.
Performance Accuracy	Provides great accuracy with a small rule set, ensuring consistent credit risk evaluations.	High accuracy, particularly in noisy environments, ensures reliable credit risk calculations.
Scalability	Scales well for large credit datasets while preserving performance efficiency.	Scalability is good, however performance may suffer significantly when dealing with very large datasets in comparison to RIPPER.
Decision Support	Clear rules help in decision-making and regulatory compliance, increasing transparency and confidence.	Detailed rules facilitate thorough decision-making, which is useful for complex credit rating scenarios.
Adaptability to Changes	Can readily be updated with new rules if credit score factors change.	Flexible in adjusting to new rules and credit scoring conditions.

Table 1.1: Motivation for choosing the RIPPER and CN2 algorithms for credit scoring

The RIPPER and CN2 algorithms were chosen for credit scoring due to their similar strengths, which satisfy the many needs of effective credit risk assessment. RIPPER focuses at transparency and efficiency, developing simple and clear rules that make basic decisions and ensure regulatory compliance. Its fast processing speeds guarantee prompt evaluations even with enormous datasets, making it perfect for those dealing with massive amounts of credit data. RIPPER's iterative pruning method effectively reduces overfitting and controls noisy data to a moderate degree, resulting in uniform and reliable results in a variety of settings.

1.1 Ethical, societal and sustainability aspects

1.1.1 Ethical considerations

The dataset [2] we picked for our study is freely available to anybody and is not restricted to a specific country. However, it is important to note that this dataset contains personal information. Even if the data is publicly available, we are committed to treating it with the utmost care and regard for privacy. This dataset includes plenty of information about the numerous elements that influence credit scores. These elements include an individual's annual income, interest rates, the number of credit cards and bank accounts they own, the types of loans they have, how many loans they've taken out, monthly balances, and total monthly loan payments.

It's important to remember that each entry in the collection represents a specific person's credit profile. This means that the data is directly related to individuals and their financial conditions. Given the importance of this information, we are dedicated to handling it professionally and ethically, with privacy as our top concern. Throughout our research, we will make every effort to ensure that the personal information contained in the dataset is treated with strict secrecy and used solely for the purposes specified in our study. We will follow data security and privacy best practices to protect the identities and privacy of the persons represented in the dataset.

1.1.2 Societal considerations

The use of RIPPER and CN2 algorithms in identifying and reducing biases in personal loan credit scoring systems is critical. This attempt is essential for ensuring that credit assessment conclusions are fair and equitable, encouraging opportunities for all, and reducing the possibility of discriminatory behaviors within lending operations.

The primary purpose of this research is to gain a better understanding of how rule-based models work, specifically the RIPPER and CN2 algorithms, to improve the accuracy and fairness of credit scoring procedures. Using these algorithms, we hope to uncover and address any inherent biases in the data or the decision-making pro-

cess. This effort not only helps to enhance credit scoring procedures but also coincides with the broader goal of promoting fairness and diversity in the financial system.

1.2 Aim and objectives

1.2.1 Aim

The thesis aims to compare RIPPER and CN2 algorithms to determine which algorithm accurately predicts personal loan credit scoring through model evaluation metrics such as accuracy, precision, recall, and F1 score.

1.2.2 Objectives

- Preprocess the dataset by handling missing values, reducing duplicates, data encoding, and data normalization to improve its quality for detailed analysis.
- Train the models using RIPPER and CN2 algorithms and evaluate each algorithm's performance against a set of evaluation metrics such as recall, accuracy, precision, and F1 score.
- Validate the model's performance through K-fold cross-validation on the training data to ensure consistency.
- Determine which algorithm, RIPPER or CN2, is the most accurate for predicting personal loan credit scores.

1.3 Research questions

RQ. Which algorithm, RIPPER or CN2, is more effective based on each evaluation metric like accuracy, precision, recall, and F1 score, in the context of personal loan credit scoring?
Motivation: The goal of this question is to thoroughly evaluate the effectiveness of the RIPPER and CN2 algorithms using a variety of evaluation metrics like F1 score, accuracy, precision, and recall. Through a comparison of these metrics, the thesis aims to clarify which algorithm performs better for personal loan credit scoring, advancing knowledge, and directing decision-making in this area.

1.4 Scope

The scope of this thesis is to analyze rule-based models in the domain of personal loan credit scoring, with a focus on comparing the RIPPER and CN2 algorithms. It consists of several important processes, beginning with the collection and preparation of relevant loan application data. Following that, both the RIPPER and CN2 algorithms will be used to create rule-based models, utilizing approaches such as cross-validation to ensure robust evaluation. The comparative analysis will look at a variety of evaluation metrics, including accuracy, recall, precision, and F1 score to determine the relative effectiveness of each technique. The thesis seeks to provide useful insights into the efficiency of rule-based models in personal loan credit scoring, contributing to both theoretical discussions and practical implementations in the financial sector.

1.5 Outline

- Chapter 1: an overview of the research topic, research questions, aims, and study importance.
- Chapter 2: A summary of the essential ideas, concepts, and frameworks that constitute the foundation for the research.
- Chapter 3: Literature review, critical evaluation of previous studies, and identification of knowledge gaps.
- Chapter 4 discusses research methodology, which includes collecting data, research methodologies, and research design.
- Chapter 5: Presentation and discussion of the research findings.
- Chapter 6: A discussion of how to answer research questions and connect outcomes with expectations.
- Chapter 7: Conclusion, review of the findings, and recommendations for further research.

Chapter 2

Background

The background section of a thesis provides context for the research by explaining the topic's historical evolution, current status, and importance. It defines the problem being addressed, justifies the chosen approach and objectives, and establishes the study's importance and goals.

Personal loan credit scoring involves predicting financial risk in financial services for customers. Organizations utilize credit and behavioral scoring to determine the creditworthiness of candidates. While credit scoring assesses eligibility for new applications, behavioral scoring informs decisions about existing clients, such as credit limit extensions or late payment resolution. Credit scoring attempts to categorize consumers as "good" or "bad" for lending purposes, with banks aiming to give credit to profitable customers while avoiding losses from challenging ones [39].

2.1 Rule-based classifiers

Rule-based classifiers use a collection of "if-then" rules to match antecedents and consequent [41]. A rule is often expressed in the following format:
IF condition, **THEN** conclusion.

The condition on the left side of the rule is also known as the antecedent. The right-hand side of the rule is known as the consequent, and it contains the class variable. During the training phase of a rule-based algorithm, a set of rules is developed to capture the patterns and correlations found in the data. These rules serve as suggestions for the classification phase, during which new instances are assigned class labels. A critical component of this approach is identifying rules triggered by specific test instance rules whose conditions correspond to the characteristics of the instance being tested [4].

When a test instance meets the logical conditions provided in the antecedent of a rule, the rule is considered activated. However, in some cases, the same test instance may activate numerous rules with contradicting consequences. This scenario necessitates strategies for resolving disputes in class label predictions, giving a consistent and correct classification result. Rule sets may satisfy one or more of the following properties:

- **Mutually exclusive rules:** This feature indicates that each rule applies to a different, non-overlapping subset of the data. As a result, a single test instance can activate no more than one rule. Decision trees frequently produce mutually exclusive rules.
- **Exhaustive rules:** Exhaustive rule sets cover the full data space, ensuring that every test case activates at least one rule. Decision trees automatically generate extensive rules. Crafting an exhaustive rule set frequently entails constructing a catch-all rule that applies the dominant class label to instances that are not covered by other rules.

Maintaining a balance between two properties' mutual exclusivity and exhaustiveness is critical for successful rule-based classification. It ensures that classification decisions are both accurate and straightforward, which improves the algorithm's performance across a variety of datasets.

When rule sets have the characteristics of being mutually exclusive and exhaustive, classification becomes easy. Each test instance corresponds to a single rule, which eliminates conflicts in class predictions caused by many rules. However, in circumstances where rule sets lack mutual exclusivity, conflicts emerging from numerous activated rules for a test instance must be resolved. In such instances, there are two main techniques for resolving conflicts:

- **Rule ordering:** Rules are systematically arranged using preset criteria, with rules often being prioritized depending on their performance metrics. This ordered structure, known as a decision list, allows for more efficient classification. When the top triggered rule encounters a test instance, it decides the appropriate class label. Any future triggered rules are ignored, with a default catch-all class applied in the absence of a triggered rule [11].
- **Unordered rules:** Alternatively, rules are left unsorted, and classification is based on the prevailing class label of all activated rules. This strategy provides robustness because it does not rely on a single rule set by a prioritizing scheme. While the training phase of unordered techniques is often efficient since all rules can be retrieved at the same time, the testing phase may be more computationally expensive due to the need to compare test examples to all rules [11].

Overall, the choice of the rule-ordering approach has a considerable impact on the efficiency and accuracy of rule-based classification algorithms, taking into account the structure of the dataset and the application domain's special requirements.

2.2 Rule induction

Rule induction, or rule learning, refers to the process of learning rules directly from data. In many rule induction systems, sequential covering is a common learning approach used. This method seeks to sequentially build a list of rules, also known as a decision list. The sequence of rules in this list is critical since it affects the precedence of rules in classification [13].

Sequential covering works by learning rules one at a time to cover all of the training data. A rule is considered to cover a data instance, whether from the training or test dataset if it meets the rule's requirements. As a result, covering the complete training data means ensuring that each training instance meets the requirements of at least one rule in the decision list [35]. It is vital to note that instances may satisfy many rules, and each rule can apply to multiple instances. The fundamental learning phase in sequential covering is to remove the training instances covered by a learned rule from the dataset. This approach guarantees that the ensuing rules capture the patterns and relationships found in the remaining data. This iterative learning process continues until either the remaining training set has been exhausted or no new rules can be extracted from the data.

In essence, sequential covering supports the systematic identification of rules from data, allowing for the creation of a decision list that accurately depicts the dataset's underlying patterns and correlations. By iteratively refining the rules based on the covered examples, this approach assures that a rule-based classifier is comprehensive and interpretable.

2.3 Sequential covering algorithms

In rule-based classification, sequential covering methods like CN2, and RIPPER [14] are frequently used to generate ordered rule lists. This method iteratively generates rules from the training dataset until a predetermined stopping criterion is met. The method normally consists of two basic steps:

- **Learn-one-rule:** The algorithm chooses a specific class label and then determines the "best" rule from the current training cases associated with that class label. This rule is then placed at the bottom of the ordered rule list. Several criteria are used to determine the "best" rule, including rule correctness, coverage, and information gain.

- **Prune training data:** Once a rule has been learned and added to the rule list, the algorithm deletes the training examples covered by that rule from the dataset. This pruning stage eliminates instances that meet the learned rule's antecedent, regardless of whether their class labels fit the rule's consequent. By deleting covered cases, the algorithm iteratively refines the rule list before focusing on learning new rules to cover the remaining data.

Sequential covering approaches [9] provide a systematic approach to rule learning by enabling the generation of ordered rule lists that prioritize informative rules for categorization. This iterative method ensures that rules are adapted to capture various patterns and relationships in the data, resulting in a comprehensive and understandable rule set. Furthermore, by pruning the training data after each rule is learned, these approaches successfully manage the rule list's complexity while also preventing overfitting. Overall, sequential covering algorithms provide a strong framework for rule-based categorization, allowing for more accurate and efficient decision-making in a variety of applications.

2.4 RIPPER algorithm

2.4.1 Learn-one-rule

RIPPER [20] relies on the Learn-One-Rule approach for iterative rule induction. Initially, the algorithm starts with an empty rule and gradually adds conditions (conjuncts) to refine it. The FOIL (First Order Inductive Learner) serves as the foundation for rule learning and classification [8]. At each phase, RIPPER chooses the best conjunct based on FOIL's Information Gain, a measure that evaluates the utility of adding new conditions. This metric analyzes the increase in classification accuracy caused by the inclusion of a condition and favors those that improve the rule's ability to effectively categorize occurrences. By maximizing information gain, RIPPER assures that each additional condition improves the rule's ability to catch positive examples while minimizing negative misclassification [5].

2.4.2 Rule pruning

RIPPER uses rule pruning to improve the quality and effectiveness of its rule set. After creating rules, RIPPER assesses their performance using a metric based on the number of positive and negative instances covered by each rule inside the validation set. This metric, defined by the formula (P - N) / (P + N), where P is the number of positive examples and N is the number of negative examples covered by the rule, measures the rule's effectiveness. If the computed measure indicates poor performance, implying that the rule could lead to misclassification or overfitting, RIPPER removes it from the rule set. By deleting ineffective rules, RIPPER assures that the final rule set remains accurate and generalizable.

2.5 CN2 algorithm

2.5.1 Learn-one-rule

The CN2 algorithm's [12] approach to rule generating is based on the Learn-One-Rule technique, just like RIPPER. Starting with an empty rule, CN2 gradually constructs rules by appending conditions to the rule antecedent. To increase the coverage of

positive examples in the rule and minimize the inclusion of negative examples, CN2 chooses the best condition at each iteration based on predetermined criteria. CN2 makes sure that every addition improves the rule's capacity to correctly categorize instances by continually improving it.

2.5.2 Rule pruning

The CN2 algorithm does not specifically include a rule pruning mechanism in its rule induction process, in contrast to RIPPER. CN2, on the other hand, does not use explicit pruning algorithms and instead concentrates on incremental rule development. CN2 uses a heuristic function to end the search process during rule generating, even though it does not prune rules in the same way as RIPPER. Without the need for explicit rule pruning, this heuristic function guarantees that the algorithm effectively searches the search space and produces accurate rule sets [34].

In essence, both the RIPPER and CN2 algorithms use the Learn-One-Rule technique for iterative rule generation, emphasizing rule refinement to appropriately identify instances. While RIPPER employs explicit rule pruning to improve the quality of its rule set, CN2 uses incremental rule development without explicit pruning procedures, instead relying on heuristic functions to assure efficiency and accuracy in rule production.

2.6 Data preprocessing and model evaluation techniques

2.6.1 Label encoding

By giving each category a unique numerical label, the process of label encoding transforms categorical data into a numerical representation. This conversion makes it easier for machine learning algorithms to process the data by allowing them to use categorical variables. But it's important to remember that, if handled improperly, label encoding may produce ordinality where none previously exists, which could affect model performance. Samples that include missing data are removed, and label encoding is used to convert the data [40].

2.6.2 Min-Max scaler

The Min-Max scaler [16] is a popular preprocessing approach for scaling characteristics to a given range, usually between 0 and 1. It operates by subtracting a feature's minimum value and then dividing it by its range (maximum minus minimum). This normalization guarantees that all features are on the same scale, preventing certain features from exceeding others due to their bigger magnitudes.

2.6.3 Train-test-split

The train-test split [37] is a fundamental strategy for evaluating the performance of a model. It is a procedure that divides the given dataset into two subsets: training and testing.
Training Set: This subset of the data is used to train the model, allowing it to discover patterns and relationships in the data.
Testing Set: This set is maintained apart from the training process and is used to assess the model's performance.

2.6.4 Validation

In machine learning, validation [29] is the process of determining how effectively a trained model generalizes to new, previously unknown data. It entails dividing the available data into training and validation sets. The training set trains the model, whereas the validation set evaluates its performance. Validation aids in detecting overfitting, which occurs when a model learns to recall the training data rather than generalize from it. The choice of validation method is determined by criteria such as dataset size, available system resources, and the desired consistency of model evaluation.

2.6.5 K-Fold cross validation

K-fold cross-validation [32] is a technique for evaluating the performance of machine learning models. The dataset is divided into k equal-sized folds, one for testing the model and the remaining (k-1 folds) for training. The process is performed k times, each using a different fold as the test set. The performance measures are then averaged over all k iterations to produce a single evaluation metric for the model. K-fold cross-validation provides a more accurate estimate of the model's performance and lowers the danger of overfitting or underfitting when compared to a single train-test split.

2.6.6 Hyperparameter tuning

Hyperparameter tuning [28] is the process of determining the best set of parameters for a machine-learning model. These hyperparameters, which drive the learning process, have a substantial impact on the model's performance. The tuning procedure typically entails picking a set of hyperparameters, training the model with various combinations, and assessing each configuration using a performance metric. Grid search, random search, and Bayesian optimization are popular methods for effectively exploring the hyperparameter space and determining the best combination that maximizes the model's performance on a validation dataset. Proper hyperparameter adjustment can significantly increase a model's accuracy and generalizability.

2.7 Performance Metrics

2.7.1 Confusion matrix

The confusion matrix isn't a standalone performance indicator, its elements are essential for assessing how successful an algorithm is. In essence, the confusion matrix offers a matrix-formatted, structured representation of the classification findings. True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) are its four main constituents [45].

- **True Positives (TP):** These are instances in which the model accurately predicts the positive class. In other words, the system correctly finds positive cases.
- **True Negatives (TN):** TN represents instances in which the model accurately predicts the negative class. In this scenario, the system accurately classifies cases as negative when they are negative.
- **False Positives (FP):** FP occurs when the model wrongly predicts a positive case. It indicates that the system incorrectly labels negative instances as positives.
- **False Negatives (FN):** FN indicated instances where the model incorrectly predicts the negative class. In this situation, the system incorrectly identifies positive instances as negatives.

Accuracy: Accuracy indicates the proportion of correctly classified instances among the total classified instances [43] .

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

Precision: Precision measures the proportion of true positive predictions (TP) divided by the sum of true positive predictions and false positive predictions [31].

$$Precision = \frac{TP}{TP + FP}$$

Recall: Recall measures the proportion of true positives correctly identified from all positive instances [17].

$$Recall = \frac{TP}{TP + FN}$$

F1 score: The F1 score is computed by combining precision and recall to assess the overall accuracy of the algorithm [10].

$$F1 = 2 \times \frac{precision \times recall}{precision + recall}$$

Chapter 3

Related Work

In 2016, **Leong** [24] compared three credit risk scoring algorithms through experimentation and those are logistic regression, neural networks, and Bayesian networks, using metrics such as accuracy, sensitivity, and precision across four datasets: balanced, imbalanced, large imbalanced, and large imbalanced with a rejection sample. In a balanced small sample, the Bayesian network model showed an accuracy of 0.77, In an unbalanced small sample, both Bayesian network and logistic regression models achieved high accuracy (0.91), with the neural network demonstrating the best precision. In an unbalanced large sample with a rejection sample, logistic regression scored an accuracy of 0.94, while the Bayesian network significantly outperformed with an accuracy of 0.98. The Bayesian network model consistently demonstrated strong performance, particularly excelling in accuracy, sensitivity, and precision, especially in scenarios of class imbalance. The neural network also performed well but lacked interpretability, while logistic regression showed comparatively lower predictive performance.

In 2020, a study conducted by **Wang et al.** [44] the effectiveness of machine learning-based credit risk models was compared using bank loan data through experimentation. Five machine learning classifiers were evaluated: Naive Bayesian Model, Logistic Regression Analysis, Random Forest, Decision Tree, and K-Nearest Neighbor Classifier. Various evaluation metrics assessed the model's performance, including accuracy, precision, recall, and F1 score. The study tested the performance of these methods using two different proportions of the testing dataset (0.2 and 0.3). Across both proportions, Random Forest exhibited the highest accuracy, with scores of 96.53% and 94.57%. The decision Tree followed with accuracy around 94.68% and 92.11%, while the K-Nearest Neighbor Classifier scored approximately 82.27% and 81.51%. Logistic Regression maintained a consistent accuracy of around 81.01% across both datasets and Naive Bayes showed similar performance, around 79.99%. The results indicated that Random Forest outperformed all other methods regarding prediction performance.

In their 2022 study, **Liu et al.** [26] introduce AugBoost-RFS and AugBoost-RFU, two novel credit scoring algorithms based on gradient-boosted Decision Trees (GBDTs). The paper evaluates various credit scoring algorithms, including AdaBoost, Decision Trees, Nearest Neighbors, Logistic Regression, Linear Discriminant Analysis, Neural Networks, Support Vector Machines, Random Forests, and GBDT, alongside AugBoost-based approaches like AugBoost-RP, AugBoost-PCA, AugBoost-NN,

AugBoost-RFU, and AugBoost-RFS through experimentation. Utilizing four datasets (BankFear, LC, Fannie, and Give) and metrics such as Accuracy, AUC, Precision, Recall, Brier loss, and H-measure, the study finds that AugBoost-RFU and AugBoost-RFS outperform other models on the BankFear and LC datasets. AugBoost-RFS slightly edges out AugBoost-RFU, suggesting supervised step-wise augmented AugBoost is superior in credit scoring. Although AugBoost-NN achieves a slightly higher AUC, AugBoost-RFS and AugBoost-RFU are deemed better choices for efficient credit scoring considering performance and efficiency. SVM proves unsuitable for large-scale credit scoring, while KNN struggles with imbalanced learning. Overall, tree-based ensemble algorithms, such as DT, AdaBoost, RF, and GBDT, notably enhance credit scoring performance.

In their 2022 study, **Pimcharee and Surinta** [33] focus on personal loans and employ data mining tools to identify key factors influencing bank loan decisions. They compare data mining procedures with and without feature selection strategies, using decision trees, multi-layer perceptrons (MLP), and support vector machines (SVM) through experimentation. Evaluation metrics such as Accuracy, Precision, and Recall are utilized. Despite experimenting with feature selection techniques like Chi-square and information gain, they find that using all 14 features yields higher accuracy rates compared to feature selection methods. Using 1,000 instances of K-Fold cross-validation, they discover that SVM performs better than MLP and decision trees, attaining the greatest accuracy rate of 97.12%, which is a significant improvement over MLP's accuracy of 83.12%. Decision trees prove less effective for personal loan evaluation, achieving only a 60.06% accuracy rate. Consequently, SVM demonstrates superior accuracy compared to MLP and decision trees, making it the preferred method for personal loan evaluation.

In 2023, **Bhattacharya and Datta** [6] compared the performance of three machine learning models: Logistic Regression, Decision Tree, and Random Forest, for predicting the diabetic condition of patients through experimentation. Diabetes and pre-diabetes were prevalent diseases, and accurately predicting them was the primary focus of the study. The research aimed to develop an algorithm to extract decision rules from the Decision Tree classifier in a human-readable format (IF-THEN rules). Additionally, logistic regression models and the tree structure of the Random Forest model were developed to classify the diabetic condition. The performance of all machine learning classifier models was measured in terms of various metrics such as recall, precision, accuracy, and F1-score using a confusion matrix. The context indicated that the Decision Tree Classifier (0.78), with the tuning of its hyperparameters, provided better accuracy compared to the Random Forest (0.75) and Logistic Regression models (0.77).

In 2021, **Tripathi et al.** [40] presented comparative evaluations and the influence of feature selection methods combined with classification methodologies for credit scoring through surveys. Several feature selection strategies, including ILFS, ECFS, Relief, FSV, LS, MFCS, UDFS, LLCFS, and CFS, were tested with classifiers including LDA, NB, TDNN, KNN, DT, ELM, RF, PART, MLP, RBFN, SVM-R, SVM-P, LRA, and SMO. Validation was performed using seven real-world credit scoring

datasets, and the results were evaluated in terms of classification accuracy, sensitivity, specificity, and G-measure. The study discovered that SVM-L and SVM-R had the highest classification accuracy on Australian and German categorical datasets, respectively. Multi-objective GA-based fuzzy classification rule extraction worked well on Australian and Japanese datasets. NRS-based feature selection and layered ensemble classification produced promising results on Australian and German categorical datasets. TDNN outperformed the other classifiers, followed by RF, on the majority of credit-scoring datasets. Overall, UDFS demonstrated consistent performance gain across different classifiers. The hybrid strategy used in the study, which included feature selection and classification, performed well, with NRS-based feature selection and layered ensemble classification outperforming the other methods. Finally, the study concluded that ensemble classification algorithms, when paired with feature selection, have the potential to attain the highest classification accuracy across the majority of datasets.

In 2019, **Munkhdalai et al.** [30] compared a variety of machine-learning algorithms, including Logistic Regression (LR), Multivariate Adaptive Regression Splines (MARS), Support Vector Machine (SVM), Random Forest (RF), Extreme Gradient Boosting (XGBoost), and Artificial Neural Network (ANN) through experimentation. Additionally, two variable-selection algorithms were employed for extracting informative features from the high-dimensional social survey data. These algorithms included the two-stage filter feature selection (TSFFS) method and a random forest-based new approach (NAP) for variable selection. The comparison showed that machine-learning methods outperformed FICO credit scoring in the 2001s. Deep neural networks and XGBoost models outperformed other machine learning algorithms in terms of accuracy and AUC. The XGBoost model had the highest accuracy but did not do well for total ECL. While accuracy is a good criterion for measuring classification performance, it does not adequately assess the capacity of the credit scoring model. AUC is inconsistent with continuous ECL. Theoretical performance measures are not suitable for evaluating credit scoring models. As a result, it's critical to use business metrics.

Rule-based classifiers provide transparency and interpretability by producing human-readable rules that explicitly describe the decision-making process. They are generally simple to implement and maintain, allowing for flexibility in responding to changing requirements. These classifiers can effectively handle missing values and domain-specific information, allowing experts to enter their insights directly into the system. Rule-based classifiers also provide explicit explanations for their predictions, which promotes user trust and understanding. They can also be built to be resistant to noise and outliers in data, resulting in more accurate predictions. Overall, rule-based classifiers are useful tools in domains requiring transparency, interpretability, and the inclusion of domain knowledge.

3.1 Research Gap

Several studies have been conducted on personal loan credit scoring. While the studies mentioned focus on machine learning classifiers such as logistic regression, neural networks, Bayesian networks, random forests, and support vector machines, they do not delve into rule-based models extensively. However, there is a research gap concerning rule-based models like RIPPER and CN2 offer interpretability and transparency, which are crucial factors in credit scoring applications, especially for regulatory compliance and risk assessment. Although Leong [24], Wang et al. [44], Liu et al. [26], Pimcharee and Surinta [33], Tripathi et al. [40], and Munkhdalai et al. [30] have explored different techniques and models for personal loan credit scoring, none of them focus on rule-based models in personal loan credit scoring with RIPPER and CN2 algorithms.

Certainly, researchers have investigated several algorithms, including the Naive Bayesian Model, Logistic Regression Analysis, Random Forest, Decision Tree, and K-Nearest Neighbor Classifier, to predict creditworthiness in the context of personal loan credit scoring. However, a clear gap appears in the analysis of two specific algorithms: RIPPER and CN2. When compared to their variations, these rule-based models provide an alternative method of credit scoring, but they did not receive as much attention in research. By examining the complexities of the RIPPER and CN2 algorithms in the context of personal loan credit scoring, this study aims to fill that gap. The research aims to identify these unused models' possible advantages, disadvantages, and special skills in determining an applicant's ability to fulfill a personal loan. This effort benefits both buyers and sellers in the end because it improves our understanding of credit scoring models and offers helpful data that may improve the precision and effectiveness of loan options.

Chapter 4

Method

The thesis's chosen technique is an experimentation methodology to achieve the aim and address the research questions. The dataset, obtained via Kaggle [2], included information about personal loan credit scores. The experiment included comparing two machine learning (ML) algorithms: RIPPER and CN2. The selected algorithms were trained using training data and validated using validation data. Their performance was then evaluated on testing data using metrics such as accuracy, precision, recall, and F1 score. Preprocessing of the data involved label encoding of categorical variables, handling missing and null values, data normalization, removal of unwanted features, and outlier detection. The data was split into three parts - training, validation, and testing datasets - using the train-test-split technique with a ratio of 60:20:20. The selected ML models underwent training and validation on the data, with hyperparameter tuning performed to enhance their performance. Model validation was conducted on each selected ML model. Subsequently, model evaluation was carried out on an optimal model with optimal parameters that had been validated beforehand, resulting in the acquisition of model evaluation metrics such as accuracy, precision, F1 score, and recall.

The experimentation procedure starts with Building and comparing models for personal loan credit scoring involves several key steps. First, necessary data is gathered and preprocessed to ensure cleanliness and suitability for training. The data is then split into training and testing sets. The models are trained using the RIPPER and CN2 algorithms, followed by hyperparameter tuning to enhance performance. Model performance is evaluated using metrics like accuracy, precision, recall, and F1 score. Validation is done through methods such as k-fold cross-validation. Finally, the results of the two algorithms are compared to determine which performs better for credit scoring is represented in the diagram shown in Figure 4.1 below.

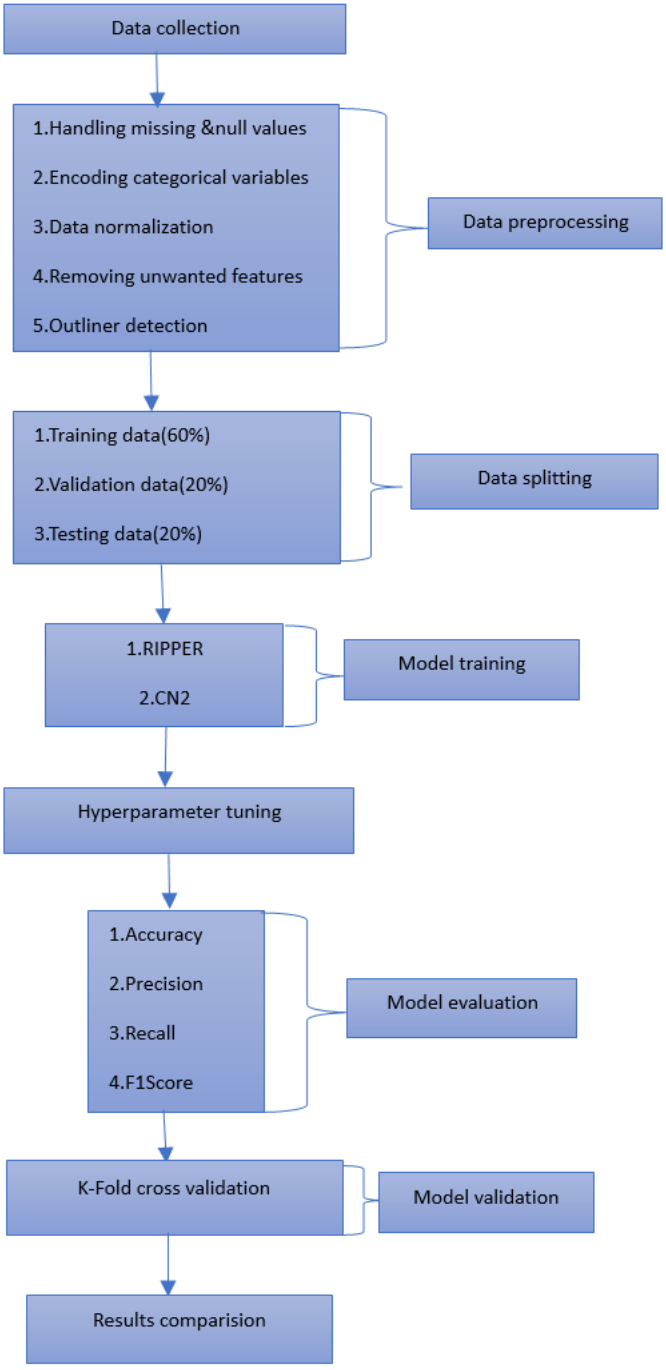


Figure 4.1: Experimentation procedure

4.1 Working environment

4.1.1 Libraries

Numpy: NumPy [18] is an essential library for machine learning, providing efficient array operations and mathematical algorithms. It is required for tasks like data manipulation, preprocessing, and numerical computations, and it provides the speed and versatility needed to efficiently implement algorithms. Its smooth interface with other ML libraries, such as scikit-learn and TensorFlow, makes it essential for developing and deploying machine learning models.

Pandas: Pandas [7] is an important Python module in machine learning for data manipulation and analysis. Pandas are used in machine learning workflows to perform data preprocessing, cleaning, and transformation, making it easier to prepare data for model training. Its ability to handle missing values, merge datasets, and execute statistical computations is extremely useful in developing strong machine-learning pipelines.

Scikit-learn: Scikit-learn [42] is an essential Python library for machine learning tasks. It offers a diverse set of techniques and tools for developing and assessing machine learning models. Scikit-learn provides a full range of functions, including classification, regression, clustering, and dimensionality reduction. It also offers utilities for preparing data, selecting features, and evaluating models.

Matplotlib: Matplotlib [38] is an important Python package for data visualization in machine learning. It provides a variety of plotting techniques and customization possibilities, allowing users to construct effective visualizations for successful data exploration and communication. Matplotlib is commonly used in machine learning processes to show data distributions, model performance metrics, feature significance, and decision boundaries, among other things. Its versatility and ease of use make it a popular tool for creating high-quality plots and charts.

Seaborn: Seaborn [23] is a powerful Python package developed on top of Matplotlib that is specifically designed for statistical data visualization in machine learning. It provides a high-level interface for constructing visually appealing and useful statistical graphs. Seaborn makes it easier to create complex visualizations including scatter plots, bar plots, heatmaps, and pair plots, allowing users to easily examine data linkages and trends. Seaborn is extensively used in machine learning projects for displaying datasets, model performance, and statistical analysis since it supports categorical variables, automatically aggregates data, and has simple customization possibilities.

Orange: The Orange library, [15] an open-source data mining and machine learning toolkit, provided an adaptable framework for investigating rule-based models in machine learning. It provided various data analysis-specific functions, such as data preparation, feature selection, and model creation. Users could use Orange's graphical interface to analyze data, visualize patterns, and create rule-based models.

Wittgenstein: The Wittgenstein library [1]is a Python module developed for rule learning and induction. It provided functions designed exclusively for rule-based modeling, allowing users to generate interpretable classification rules from their data. The library assisted operations like rule generation, rule evaluation, and model interpretation, allowing users to get insights into their datasets and create transparent machine-learning models. Overall, the Wittgenstein library contributed significantly to the advancement of machine learning by providing a robust set of tools for rule learning and induction.

4.2 Data collection

The Credit Score Classification dataset used in this thesis was collected from Kaggle and is publicly available. It includes credit score information, as well as the applicant's personal and financial characteristics. This dataset [2] contains 28 variables, with 27 features and 1 target variable. The target variable in the data is credit_score each of which is discussed below in Table 4.1.

S.No	Attributes	Description
1	ID	A unique number assigned to each record.
2	Customer_ID	A special number for each customer, helping to connect different pieces of information about the same person.
3	Month	The specific time when the data was collected or when credit scores were calculated.
4	Name	The person's name.
5	Age	How old the person is.
6	SSN	A unique number used to identify a person.
7	Occupation	The job a person has, which can tell us about their financial situation.
8	Annual_Income	How much money a person makes in a year.
9	Monthly_Inhand_Salary	The amount of money a person takes home each month after taxes.
10	Num_Bank_Accounts	How many bank accounts does a person have?
11	Num_Credit_Card	The number of credit cards a person owns.
12	Interest_Rate	The cost of borrowing money, if someone has a loan or credit.
13	Num_of_Loan	How many loans a person has.
14	Type_of_Loan	What kind of loan does someone have, like a student loan or a car loan?
15	Delay_from_due_date	How late someone is on paying their bills.
16	Num_of_Delayed_Payment	How many times someone has missed a payment.
17	Changed_Credit_Limit	Whether a person's credit limit has been adjusted.
18	Num_Credit_Inquiries	How many times lenders have checked someone's credit?
19	Credit_Mix	The different kinds of credit someone uses, like credit cards or loans.
20	Outstanding_Debt	How much money someone still owes.
21	Credit_Utilization_Ratio	How much of someone's available credit they're using.
22	Credit_History_Age	How long someone has had credit for.
23	Payment_of_Min_Amount	Whether someone pays the minimum amount due on their bills.
24	Total_EMI_per_month	The total amount someone pays in monthly installments.
25	Amount_invested_monthly	How much someone saves or invests each month.
26	Payment_Behaviour	How someone typically handles paying their bills, like consistently on time or occasionally late.
27	Monthly_Balance	How much money someone has left in their account at the end of each month.
28	Credit_Score	A number that represents how good someone is at managing credit.

Table 4.1: Description of attributes in the dataset

The dataset is loaded using the panda's library's "pd.read_csv" method, which is efficient with CSV file formats. The "df.head" function provides a rapid summary of the dataset's structure and contents. This function provides a short screenshot of the DataFrame, displaying the first few rows and their appropriate column names as shown in Figure 4.2.

	ID	Customer_ID	Month	Name	Age	SSN	Occupation	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	...	Credit_Mix
0	5634	3392	1	Aaron Maashoh	23.0	821000265.0	Scientist	19114.12	1824.843333	3.0	...	Good
1	5635	3392	2	Aaron Maashoh	23.0	821000265.0	Scientist	19114.12	1824.843333	3.0	...	Good
2	5636	3392	3	Aaron Maashoh	23.0	821000265.0	Scientist	19114.12	1824.843333	3.0	...	Good
3	5637	3392	4	Aaron Maashoh	23.0	821000265.0	Scientist	19114.12	1824.843333	3.0	...	Good
4	5638	3392	5	Aaron Maashoh	23.0	821000265.0	Scientist	19114.12	1824.843333	3.0	...	Good

Figure 4.2: Overview of the dataset

4.3 Data preprocessing

Data preprocessing is the process of cleaning, converting, and organizing raw data before analysis to improve the accuracy and efficiency of machine learning algorithms. The dataset for this thesis was preprocessed using a variety of techniques. This included performing label encoding on categorical variables, addressing missing and null values, normalizing the data, removing irrelevant features, and detecting outliers.

4.3.1 Handling missing & null values

Handling missing and null values involves recognizing and addressing situations where data is missing or incomplete. This includes procedures like removal, which excludes rows or columns with missing values from the analysis. The goal is to make sure that the dataset is complete and appropriate for training and testing the models. The dataset we gathered appears to be complete since there were no missing values detected.

4.3.2 Encoding categorical variables

Encoding categorical variables involves converting category data into numerical representations. This is required since many algorithms need numerical input. Certain variables in the selected dataset need to be converted to numerical representations. The variables were 'Name', 'Occupation', 'Credit_Mix', 'Type_of_Loan', 'Payment_of_Min_Amount', 'Payment_Behaviour', and 'Credit_Score'. These categorical variables were preprocessed into numerical equivalents to make them easier to use in the study. Figure 4.3 shows a diagram of how categorical data is converted

into numerical representations of variables.

	ID	Customer_ID	Month	Name	Age	SSN	Occupation	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	...	Credit_Mix
0	5634	3392	1	11	23.0	821000265.0	12	19114.12	1824.843333	3.0	...	1
1	5635	3392	2	11	23.0	821000265.0	12	19114.12	1824.843333	3.0	...	1
2	5636	3392	3	11	23.0	821000265.0	12	19114.12	1824.843333	3.0	...	1
3	5637	3392	4	11	23.0	821000265.0	12	19114.12	1824.843333	3.0	...	1
4	5638	3392	5	11	23.0	821000265.0	12	19114.12	1824.843333	3.0	...	1
...	...	...	...	...	...	...	...	...	...	...	...	...
99995	155625	37932	4	6508	25.0	78735990.0	9	39628.99	3359.415833	4.0	...	1
99996	155626	37932	5	6508	25.0	78735990.0	9	39628.99	3359.415833	4.0	...	1
99997	155627	37932	6	6508	25.0	78735990.0	9	39628.99	3359.415833	4.0	...	1
99998	155628	37932	7	6508	25.0	78735990.0	9	39628.99	3359.415833	4.0	...	1
99999	155629	37932	8	6508	25.0	78735990.0	9	39628.99	3359.415833	4.0	...	1

Figure 4.3: The image shows the conversion of category data to numerical representation

4.3.3 Data normalization

Data normalization is a preprocessing technique that reduces numerical features to a predefined range, typically 0 to 1, to ensure all features have a similar scale and prevent any single feature from dominating during model training. In this study, several variables, including 'SSN', 'Annual_Income', 'Monthly_Inhand_Debt', 'Credit_History_Age', and 'Monthly_Balance', were normalized using Min-Max scaling. This involved calculating the minimum and maximum values for each feature and then applying the Min-Max scaling formula to transform each feature's values to the 0-1 range. This scaling strategy maintained consistency across the dataset, ensuring that no individual feature skewed the model's learning process. The Min-Max Scaler, shown in Figure 4.4, normalizes numerical features within a predefined range.

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

features = ['SSN', 'Annual_Income', 'Monthly_Inhand_Salary', 'Outstanding_Debt', 'Credit_History_Age', 'Monthly_Balance']

for i in features:
  data[i] = scaler.fit_transform(data[[i]])
data
```

Figure 4.4: The picture shows how the min-max scaler changes numerical features to fit within a specific range of values

4.3.4 Removing unwanted features

Unwanted features were deleted from the dataset during the preparation step. By removing the unwanted features, the dataset was simplified, lowering complexity and perhaps improving the algorithm's performance by focusing on the most essential information. During preprocessing, the variables 'ID' and 'Customer_ID' were deleted

from the dataset. This decision was made to remove identifiers that were irrelevant to the analysis or model training process. Figure 4.5 shows the unwanted features that are removed.

```
columns_to_remove = ['ID', 'Customer_ID']

# Drop the specified columns
data.drop(columns=columns_to_remove, inplace=True)

data
```

Figure 4.5: Removing unwanted features from the dataset

4.3.5 Outlier detection

Outlier detection occurred during the data preprocessing stage. This required finding and addressing observations that differed significantly from the rest of the dataset. During the data processing, some outliers were discovered in the dataset. These outliers were data points that differed greatly from the remainder of the observations. Before outlier detection, the dataset had 1,000,000 rows. Following the outlier detection process, there were only 76,742 rows left, showing that some rows had been deleted due to outliers. This step guaranteed that the dataset was high quality and reliable enough for further research.

Outliers were found in numerous columns of the researched dataset. The investigation indicated considerable differences from the standard in a variety of financial factors. There were 2000 outliers in the "Annual_Income" column, 588 in Monthly_Inhand_Salary, and 4002 in Delay_from_Due_date. Also, Changed_Credit_Limit had 808 outliers, while Num_Credit_Inquiries and Total_EMI_per_month had 642 and 4552 outliers, respectively. and Outstanding_Debt had 5810 outliers, Amount_invested_monthly and Monthly_Balance had 3183 and 5454, outliers respectively. Figures 4.7 and 4.8 illustrate the outliers detected and the outliers before and after detection, respectively.

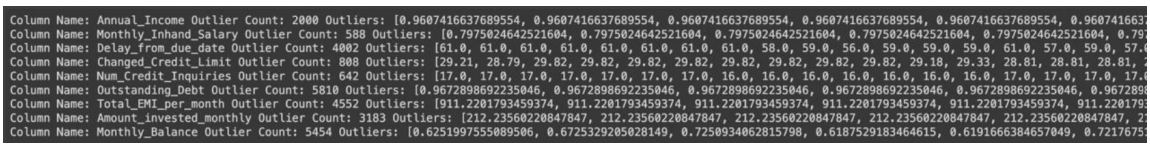


Figure 4.6: Count of the outliers detected

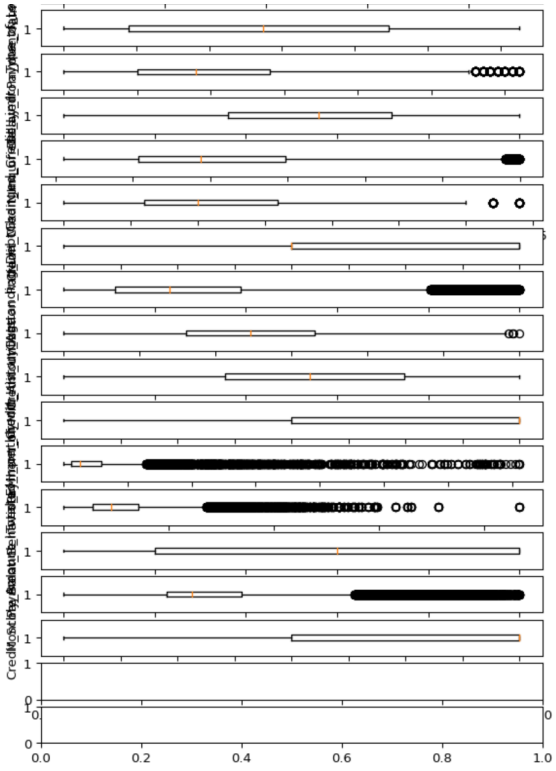


Figure 4.7: Outliers identified in the dataset

	Month	Name	Age	SSN	Occupation	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	Num_Credit_Card	Interest_Rate
0	1	11	23.0	0.820991	12	0.069997	0.102087	3.0	4.0	3.0
1	2	11	23.0	0.820991	12	0.069997	0.102087	3.0	4.0	3.0
2	3	11	23.0	0.820991	12	0.069997	0.102087	3.0	4.0	3.0
3	4	11	23.0	0.820991	12	0.069997	0.102087	3.0	4.0	3.0
4	5	11	23.0	0.820991	12	0.069997	0.102087	3.0	4.0	3.0
...	...	...	...	...	...	...	...	...	...	...
99995	4	6508	25.0	0.078662	9	0.188593	0.205072	4.0	6.0	7.0
99996	5	6508	25.0	0.078662	9	0.188593	0.205072	4.0	6.0	7.0
99997	6	6508	25.0	0.078662	9	0.188593	0.205072	4.0	6.0	7.0
99998	7	6508	25.0	0.078662	9	0.188593	0.205072	4.0	6.0	7.0
99999	8	6508	25.0	0.078662	9	0.188593	0.205072	4.0	6.0	7.0

100000 rows × 26 columns

	Month	Name	Age	SSN	Occupation	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	Num_Credit_Card	Interest_Rate
0	1	11	23.0	0.820991	12	0.069997	0.102087	3.0	4.0	3.0
1	2	11	23.0	0.820991	12	0.069997	0.102087	3.0	4.0	3.0
2	3	11	23.0	0.820991	12	0.069997	0.102087	3.0	4.0	3.0
3	4	11	23.0	0.820991	12	0.069997	0.102087	3.0	4.0	3.0
4	5	11	23.0	0.820991	12	0.069997	0.102087	3.0	4.0	3.0
...	...	...	...	...	...	...	...	...	...	...
99995	4	6508	25.0	0.078662	9	0.188593	0.205072	4.0	6.0	7.0
99996	5	6508	25.0	0.078662	9	0.188593	0.205072	4.0	6.0	7.0
99997	6	6508	25.0	0.078662	9	0.188593	0.205072	4.0	6.0	7.0
99998	7	6508	25.0	0.078662	9	0.188593	0.205072	4.0	6.0	7.0
99999	8	6508	25.0	0.078662	9	0.188593	0.205072	4.0	6.0	7.0

76742 rows × 26 columns

Figure 4.8: The picture displays the outliers both before and after their detection.

4.4 Proportion probabilities

The proportion probabilities for the selected data suggested that the dataset was balanced, with an equitable distribution of the 'good', 'standard', and 'poor' categories. The balanced representation of data categories was observed across the dataset, ensuring a fair distribution and allowing for unbiased analysis and modeling. The dataset provided information about credit mix categories, specifically standard, poor, and good, which were represented in the column labeled "credit_mix." Figure 4.9 shows a schematic diagram of the balanced proportion probabilities for given data.

$$\text{Proportionality of specified class} = \frac{\text{Number of specified class instances}}{\text{Total number of instances}}$$

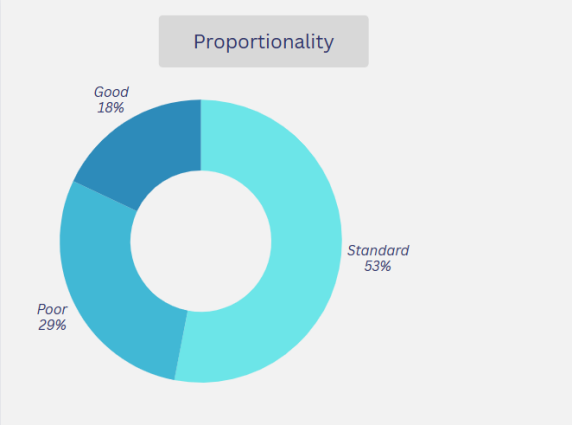


Figure 4.9: The figure represents the balanced proportion probability for the selected data.

4.5 Data splitting

After preprocessing, splitting was done. During this step, both datasets were divided into three portions, each with a 60:20:20 ratio, namely training data, validation data, and testing data. The Training data was used to train the specified algorithms, RIPPER and CN2, which were then validated using K-Fold Cross Validation. The performance of each trained model was assessed using the Testing data.

To accomplish this splitting, the 'train-test-split' approach from the 'sklearn.model_selection' package was used twice because of its limitation of dividing the data into only two parts. Initially, the entire dataset was separated into two halves with a 60:40 ratio, with 60% designated as training data and the remaining 40% containing both validation and testing data. The 40% portion was then divided into two equal sections using the same procedure for further validation and testing. Figure 4.10 depicts how the data is split.

```
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4, random_state=42)
X_test, X_val, y_test, y_val = train_test_split(X, y, test_size=0.5, random_state=42)
```

Figure 4.10: Train-test split function

4.6 Model training

After splitting, model training for RIPPER and CN2 using the credit score classification dataset was carried out methodically. Each algorithm was trained separately on the dataset. The training procedure involved providing the algorithm with labeled data and allowing it to iteratively create classification rules based on the observed patterns in the data. During this iterative process, both RIPPER and CN2 responded to the dataset's properties and refined their rule sets to attain maximum classification accuracy.

4.6.1 RIPPER

The RIPPER model training using the credit score classification dataset began with importing the RIPPER module from the Wittgenstein library. Following that, the RIPPER classifier was constructed. The classifier was then trained on the combined training data, which merged the features and target labels along the column axis. The 'Credit_Score' feature was chosen as the class feature, and 'pos_class=0' indicated a positive class label. In this study, "feature" refers to characteristics associated with loan applicants, such as income and credit history. The term "class label" refers to the algorithm-generated predicted outcome, such as loan acceptance or rejection.RIPPER evaluated the dataset throughout the training process, looking for patterns and correlations between features and class labels. After training, the RIPPER classifier was ready to make predictions on new data. The trained classifier was used to make predictions on the test data (X_test), allowing the performance of RIPPER in credit-scoring tasks to be evaluated.

4.6.2 CN2

To prepare data for training a CN2 model with the Orange library on a credit score classification dataset. Initially, a list of categorical columns (cat_cols) is defined. Then iterates through the columns of the training data, categorizing them as continuous or discrete variables. Continuous variables are represented by instances of Orange's ContinuousVariable class, whereas discrete variables are represented by instances of DiscreteVariable, which incorporate unique values taken from the training data. This step creates the scope, which effectively determines the structure and types of features, as well as the target variable used in the model.

The unique values recovered from the training data are then used to generate a discrete variable that represents the target variable "Credit_Score". With this fully defined, it converts the features and target variables to NumPy arrays, which are

then used to create an Orange Table. This table encapsulates the data in a format compatible with Orange's modeling functionalities, facilitating the subsequent training of a CN2 model for credit score classification.

4.7 Hyperparameter tuning

Following training, both the RIPPER and CN2 algorithms require hyperparameter tuning.
RIPPER: The RIPPER algorithm's hyperparameter tuning method took into consideration two primary parameters: min_support and pruning. The min_support parameter specifies the minimum number of occurrences needed to examine a rule, with values tested at 2, 3, and 5. Additionally, the pruning parameter, which governs the approach for simplifying the rule set, was tested with two reduced_error_pruning' and 'zero_gain_pruning'. The goal was to find the best configuration that maximized the RIPPER algorithm's accuracy on validation data by systematically evaluating these parameters and their corresponding values. It should be noted that all other parameters stay at their default values throughout the tuning process.

For RIPPER, min_support values of 2, 3, and 5 were tested to find a balance between capturing meaningful, less frequent rules and ensuring statistical significance. Pruning strategies such as reduced_error_pruning and zero_gain_pruning were used to reduce overfitting and simplify the rule set, hence enhancing accuracy and interpretability.

CN2: The major parameter examined in the hyperparameter tuning procedure for the CN2 method is the minimal support (min_support), which specifies the smallest number of instances needed to consider a rule during model training. The various values for min_support, such as 2 and 5, are tested. Furthermore, the choice of base rule learner is explored, including possibilities such as OneRLearner and CN2UnorderedLearner. The purpose is to identify the best set of parameters to optimize the algorithm's accuracy on validation data. All other possibilities remain at their default values.

Tuning for CN2 focuses on min_support and the basic rule learner. Min_support values of 2 and 5 were tested to find a compromise between rule granularity and robustness. Two base rule learners, OneRLearner and CN2UnorderedLearner, were compared to determine the best balance of simplicity and complexity in rule construction, to improve accuracy while maintaining interpretability.

4.8 Model evaluation

Following hyperparameter tuning, model evaluation was performed to evaluate the performance of the RIPPER and CN2 models. The dataset was utilized to train both models, which were then tuned using hyperparameters to improve their prediction skills. The best-performing model identified throughout the optimization process was then utilized to evaluate its performance on the testing data. Several evaluation metrics were produced to provide a general evaluation of the model's effectiveness.

The classification models' performance was described using a confusion matrix, which included the number of true positives, true negatives, false positives, and false negatives, providing precise insights into the types of errors produced. Accuracy, defined as the ratio of properly predicted instances to total instances, provided a summary of model performance, but it could be misleading when dealing with imbalanced data. Precision or the ratio of accurately predicted positive cases to total expected positives, was critical in situations when the cost of false positives was significant, such as credit scoring. Remember that the ratio of accurately predicted positive occurrences to all actual positives was critical in situations where missing positive instances (false negatives) had serious effects. The F1-score, which is the harmonic mean of precision and recall, provided a balanced metric that was particularly effective in circumstances of imbalanced class distribution because it compensated for both false positives and false negatives. The conclusions from these evaluations were expanded upon in the following sections of the analysis.

4.9 Model validation

Following the evaluation, model validation was conducted. The thesis used K-fold cross-validation to evaluate the performance of both the RIPPER and CN2 models. This approach divides the dataset into 'k' equal-sized folds, where 'k' is the number of folds supplied. Five folds were employed in this research to provide rigorous evaluation while maintaining computational efficiency. During each iteration, one fold was reserved as the validation set, while the remaining folds were used to train the model. This procedure was done 'k' times, with each fold serving as the validation set. The model's overall performance was carefully evaluated by averaging performance metrics such as accuracy overall folds, thus limiting the influence of data partitioning variability. Figure 4.11 presents the diagrammatic representation of the k-fold cross-validation technique.

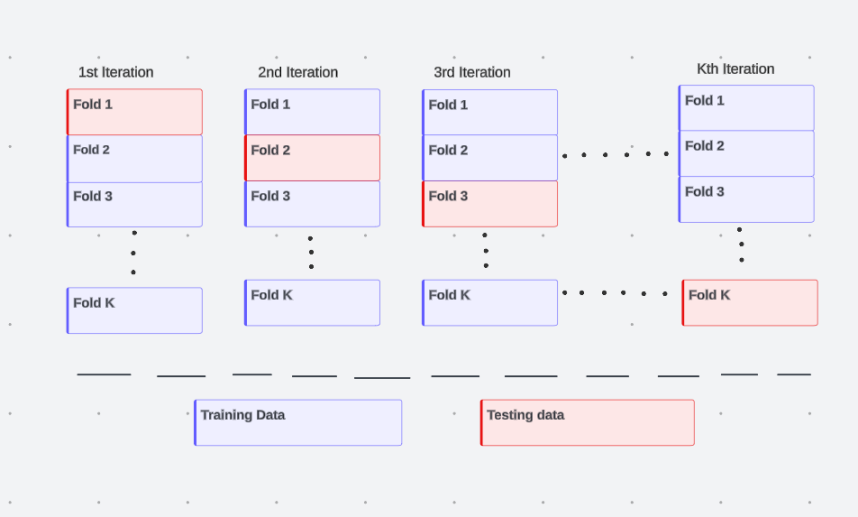


Figure 4.11: The diagram for K-fold cross-validation.

Chapter 5

Results and Analysis

RIPPER and CN2 are rule-based classification techniques. RIPPER begins with an empty rule set and adds rules repeatedly, eagerly maximizing collecting data while pruning them to minimize errors. CN2, on the other hand, creates rules using a separate-and-conquer strategy, continuously covering instances from one class at a time while dealing with noisy input. Both methods aim to produce compact, comprehensible rule sets for classification tasks, but their rule generation and refinement strategies are different.

5.1 Rules generated by RIPPER and CN2

RIPPER iteratively creates classification rules. It begins by constructing an initial rule that applies to a subset of the dataset's occurrences. The rules are then iteratively evaluated and refined, with conditions added to expand coverage while preserving or enhancing accuracy. Rules are evaluated based on their contribution to categorization accuracy, and ineffective rules are removed. This iterative refinement process continues until no further improvements can be achieved or a stopping condition is satisfied. The resulting collection of rules is easy to understand and properly identifies dataset occurrences, ensuring transparency in decision-making. The rules that are generated by CN2 are shown in Figure 5.1.

```
IF Name==4529 THEN Credit_Score=0
IF Name==1303 THEN Credit_Score=0
IF Name==9338 THEN Credit_Score=1
IF Name==9818 THEN Credit_Score=0
IF Name==7325 THEN Credit_Score=0
IF Name==3705 THEN Credit_Score=1
IF Name==1233 THEN Credit_Score=0
IF Name==5692 THEN Credit_Score=0
IF Name==6194 THEN Credit_Score=2
IF Name==6026 THEN Credit_Score=1
IF Name==4087 THEN Credit_Score=0
IF Name==5091 THEN Credit_Score=2
IF Name==3326 THEN Credit_Score=0
IF Name==391 THEN Credit_Score=0
IF Name==55 THEN Credit_Score=0
IF Name==8719 THEN Credit_Score=1
IF Name==9730 THEN Credit_Score=0
IF Name==3147 THEN Credit_Score=0
IF Name==5574 THEN Credit_Score=0
IF Name==6596 THEN Credit_Score=0
IF Name==8348 THEN Credit_Score=0
IF Name==1425 THEN Credit_Score=1
IF Name==4006 THEN Credit_Score=0
IF Name==8466 THEN Credit_Score=0
IF Name==9344 THEN Credit_Score=0
IF Name==1431 THEN Credit_Score=2
IF Name==5450 THEN Credit_Score=2
IF Name==8014 THEN Credit_Score=2
IF Name==8720 THEN Credit_Score=2
IF Name==9888 THEN Credit_Score=0
IF Name==3930 THEN Credit_Score=0
IF Name==4979 THEN Credit_Score=1
IF Name==701 THEN Credit_Score=0
IF Name==8569 THEN Credit_Score=2
IF Name==1988 THEN Credit_Score=1
IF Name==4230 THEN Credit_Score=2
IF Name==5668 THEN Credit_Score=0
IF Name==7968 THEN Credit_Score=1
IF Name==4852 THEN Credit_Score=0
IF Name==6393 THEN Credit_Score=0
IF Name==9389 THEN Credit_Score=0
IF Name==3586 THEN Credit_Score=2
IF Name==20 THEN Credit_Score=2
IF Name==8031 THEN Credit_Score=0
IF Name==6504 THEN Credit_Score=2
IF Name==4823 THEN Credit_Score=2
IF Name==3110 THEN Credit_Score=0
IF Name==634 THEN Credit_Score=1
IF Name==4404 THEN Credit_Score=0
```

Figure 5.1: Rules generated by RIPPER algorithm

CN2 is a rule-based machine learning method designed for classification problems. It creates classification rules by iteratively examining the dataset for patterns and correlations between input features and class labels. CN2 builds rules step by step, beginning with the most predictive attribute and gradually adding conditions to improve classification accuracy. The algorithm repeats this process until no more improvements can be made. The generated rules are interpretable and may be represented in human-readable language (IF-THEN rules), making them useful for comprehending the decision-making process. Overall, CN2 provides a transparent and readable method of classification, making it appropriate for applications requiring interpretability. The rules that are generated by CN2 are shown in Figure 5.2.

```
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Total_EMI_per_month=>196.38^Age=33.0-37.0^Delay_from_due_date=>37.0]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Month=1^Payment_Behaviour=4^Occupation=11.0-13.0]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Month=3^Interest_Rate=20.0-27.0^Num_Bank_Accounts=>8.0^Payment_Behaviour=5]
[Credit_Mix=0^Outstanding_Debt=0.29-0.39^Month=2^Payment_of_Min_Amount=2^Total_EMI_per_month=96.41-136.36]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Month=1]
[Credit_Mix=0^Outstanding_Debt=0.29-0.39^Month=3^Age=<19.0^Num_of_Delayed_Payment=>20.0]
[Credit_Mix=0^Outstanding_Debt=0.29-0.39^Credit_History_Age=0.23-0.33^Month=1]
[Credit_Mix=0^Outstanding_Debt=0.29-0.39^Month=2^Num_Bank_Accounts=>8.0^Credit_History_Age=0.23-0.33]
[Credit_Mix=0^Outstanding_Debt=0.29-0.39^Payment_Behaviour=5^Delay_from_due_date=>37.0^Num_Credit_Card=7.0-8.0^Num_of_Loan=4.0]
[Credit_Mix=0^Outstanding_Debt=0.29-0.39^Payment_Behaviour=5^Num_Credit_Inquiries=>11.0^Credit_History_Age=0.23-0.33]
[Credit_Mix=0^Outstanding_Debt=0.29-0.39^Month=2^Payment_of_Min_Amount=2]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Month=2^Changed_Credit_Limit=<2.54^Interest_Rate=20.0-27.0]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Month=2^Amount_invested_monthly=28.93-34.69]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Age=33.0-37.0^Num_Bank_Accounts=>8.0^Num_Credit_Card=5.0-6.0]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Month=3^Num_Bank_Accounts=6.0-7.0^Payment_of_Min_Amount=2^Interest_Rate=>27.0]
[Credit_Mix=0^Outstanding_Debt=0.29-0.39^Month=3^Credit_History_Age=0.23-0.33]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Payment_Behaviour=5]
[Credit_Mix=0^Outstanding_Debt=0.29-0.39^Interest_Rate=20.0-27.0^Num_of_Delayed_Payment=17.0-19.0^Type_of_Loan=614.0-1286.0^Name=2062.0-3122.0]
[Credit_Mix=0^Outstanding_Debt=0.29-0.39^Payment_Behaviour=5]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Total_EMI_per_month=>196.38^Monthly_Inhand_Salary=0.26-0.34^Name=3122.0-4091.0]
[Credit_Mix=0^Changed_Credit_Limit=2.54-4.47^Credit_History_Age=<0.23]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Month=3]
[Credit_Mix=0^Outstanding_Debt=0.26-0.29^Delay_from_due_date=>37.0^Total_EMI_per_month=73.11-96.41]
[Credit_Mix=0^Outstanding_Debt=0.29-0.39]
[Interest_Rate=20.0-27.0^Credit_Mix=2^Payment_Behaviour=5^Num_of_Delayed_Payment=8.0-10.0^Annual_Income=0.058-0.076]
[Interest_Rate=20.0-27.0^Credit_Mix=2^Payment_Behaviour=5^Num_Credit_Inquiries=>11.0^Month=7]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Interest_Rate=20.0-27.0^Monthly_Inhand_Salary=0.34-0.44]
[Credit_Mix=0^Outstanding_Debt=0.26-0.29^Delay_from_due_date=>37.0^Num_Credit_Inquiries=6.0-8.0^Num_Credit_Card=>8.0]
[Interest_Rate=20.0-27.0^Credit_Mix=2^Month=2^Num_of_Loan=2.0]
[Credit_Mix=0^Changed_Credit_Limit=<2.54^Num_Bank_Accounts=6.0-7.0^Interest_Rate=>27.0^Monthly_Inhand_Salary=0.17-0.2]
[Credit_Mix=0^Outstanding_Debt=0.26-0.29^Total_EMI_per_month=136.36-196.38^Type_of_Loan=>5573.0]
[Interest_Rate=20.0-27.0^Credit_Mix=2^Month=1^Num_Bank_Accounts=5.0-6.0^Payment_Behaviour=5]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Total_EMI_per_month=>196.38]
[Credit_Mix=0^Outstanding_Debt=0.26-0.29^Interest_Rate=>27.0^SSN=>0.9]
[Interest_Rate=20.0-27.0^Credit_Mix=2^Month=3^Outstanding_Debt=0.29-0.39^Payment_Behaviour=5^Credit_History_Age=0.23-0.33]
[Credit_Mix=0^Outstanding_Debt=0.26-0.29^Month=3^Num_Bank_Accounts=>8.0]
[Interest_Rate=20.0-27.0^Credit_Mix=2^Month=1^Num_of_Loan=4.0]
[Credit_Mix=0^Changed_Credit_Limit=4.47-6.39]
[Interest_Rate=20.0-27.0^Credit_Mix=2^Delay_from_due_date=>37.0^Num_Credit_Inquiries=>11.0^Credit_History_Age=0.33-0.44]
[Interest_Rate=20.0-27.0^Credit_Mix=2]
[Interest_Rate=>27.0^Credit_Mix=2^Month=1^Delay_from_due_date=>37.0]
[Interest_Rate=>27.0^Credit_Mix=2^Month=3]
[Credit_Mix=0^Changed_Credit_Limit=<2.54^Credit_History_Age=<0.23^Num_Bank_Accounts=6.0-7.0]
[Credit_Mix=0^Changed_Credit_Limit=<2.54^Monthly_Inhand_Salary=0.094-0.13]
[Credit_Mix=0^Outstanding_Debt=0.26-0.29^Payment_Behaviour=4^Payment_of_Min_Amount=2]
[Credit_Mix=0^Changed_Credit_Limit=<2.54^Payment_Behaviour=4^Monthly_Inhand_Salary=0.075-0.094]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Annual_Income=0.19-0.25^Num_Bank_Accounts=>8.0^Age=<19.0]
[Credit_Mix=0^Outstanding_Debt=0.26-0.29]
[Credit_Mix=0^Outstanding_Debt=0.39-0.51^Age=26.0-30.0^Num_of_Loan=3.0]
```

Figure 5.2: Rules generated by CN2 algorithm

5.2 Results of the experiment

5.2.1 Accuracy

The study used accuracy as the major performance metric to determine the more precise algorithm among the two. The selected algorithms were evaluated for accuracy using the testing dataset, and the results were then compared. The results, shown in Table 5.1, demonstrated the models' accuracy. After examining the findings, it was clear that the CN2 algorithm exceeded the RIPPER algorithm, with a higher accuracy rate of 47.51% compared to RIPPER's 38.24

Algorithms	**Accuracy**
RIPPER(Repeated Incremental Pruning to Produce Error Reduction)	38.24%
CN2(ClassiNet 2)	**47.51%**

Table 5.1: Accuracy of the test data

5.2.2 Precision

In this study, precision was another crucial metric used to evaluate the effectiveness of the chosen algorithms. Using the testing dataset, the results were compared to the precision of both algorithms to determine which performed better. The findings of this evaluation are reported in Table 5.2, allowing for comparison. After analysis, it was discovered that there was a minor difference between the two algorithms. The CN2 algorithm displayed higher precision with an outcome of 42%, whereas the RIPPER method attained a slightly lower precision of 34%.

Algorithms	**Precision**
RIPPER(Repeated Incremental Pruning to Produce Error Reduction)	34%
CN2(ClassiNet 2)	**42%**

Table 5.2: Precision of the test data

5.2.3 Recall

Another evaluation metric used in this study was recall, which tried to determine the most successful algorithm among the chosen. The testing dataset was used to evaluate the recall of the selected algorithms and then compare their results. Table 5.3 displays the results, which allow for a direct comparison of the algorithm's recall performance. After reviewing the findings, it was discovered that the RIPPER algorithm had a higher recall of 58%, followed by CN2 with a recall of 43%.

Algorithms	**Recall**
RIPPER(Repeated Incremental Pruning to Produce Error Reduction)	**58%**
CN2(ClassiNet 2)	43%

Table 5.3: Recall of the test data

5.2.4 F1 score

Another performance metric addressed in this study was the F1 score, which was used to assess the effectiveness of the chosen algorithms. The chosen algorithms for the F1 scores are tested on the testing dataset before comparing the results. The results, described in Table 5.4, allowed for a direct comparison of the algorithm's performance based on the F1 score. After evaluating the findings, it was discovered that the CN2 algorithm had a higher F1 score of 42%, followed by RIPPER with a score of 38%.

Algorithms	**F1 score**
RIPPER(Repeated Incremental Pruning to Produce Error Reduction)	38%
CN2(ClassiNet 2)	**42%**

Table 5.4: F1 score of the test data

5.3 Analysis

The CN2 algorithm outperforms the RIPPER algorithm in terms of accuracy, due to the consistency and complexity of the rules generated. RIPPER's rules are less consistent since its iterative pruning technique, while successful at controlling noise and limiting overfitting, may result in simpler rules that may not capture the entire complexity of the data. This simplicity can lead to more errors in predictions using RIPPER, particularly in datasets with complex patterns and relationships. As a result, RIPPER's accuracy decreases since it is unable to consistently and effectively identify data in more elaborate and refined credit scoring conditions.

On the other hand, CN2 outperforms by producing more consistent and comprehensive rules. Its rules frequently incorporate longer statements with many conditions, allowing for a more nuanced and comprehensive analysis of the data. This approach enables CN2 to better analyze and predict the complexity of credit data, resulting in increased accuracy. CN2's emphasis on important features and detailed rule construction allows it to handle more diverse and complicated datasets than RIPPER. Figures 5.1 and 5.2 demonstrate the difference by displaying how CN2's rules encompass additional variables and circumstances, resulting in a more accurate and robust classification process. This improved level of detail and consistency in CN2's rules improves its capacity to grasp the complexity of the data, resulting in higher overall accuracy and performance in credit scoring applications.

RIPPER's iterative pruning strategy works well with noisy data because iteratively refining rules reduces overfitting and improves generalization. This repeated refining process enables RIPPER to gradually adapt its rule set to better fit the data, effectively reducing the effects of noise. While, CN2 focuses on gathering data during rule development, which also aids in noise reduction by prioritizing features that contribute the most to classification accuracy. While CN2 does not use pruning in the same manner that RIPPER does, its iterative classification rule generation seeks to reduce the impact of noise by focusing on the most relevant features. RIPPER's iterative pruning approach improves efficiency with noisy data by iteratively modifying rules and reducing overfitting.

However, it is important to note that the superiority of CN2 over RIPPER is not absolute and may vary depending on the datasets used for analysis. Different datasets have distinct properties such as variable distributions, feature correlations, and noise levels, all of which can have a major impact on algorithm performance. For exam-

ple, a dataset with well-separated classes may not highlight the strengths of CN2's comprehensive rules as effectively as a more complex dataset.

In summary, while CN2 typically provides superior accuracy due to its consistent and thorough rule development, it is critical to account for the variability introduced by diverse datasets. Only rigorous and diversified testing will allow us to confirm the overall superiority of one algorithm over the other in the context of personal loan credit scoring.

Chapter 6

Discussion

In the study, two algorithms, RIPPER and CN2, were carefully selected for detailed examination. The credit score classification dataset was chosen and obtained from Kaggle, a well-known platform with a varied variety of datasets. With the main goal of shedding light on the efficiency of various algorithms, the study sought to provide a vast knowledge of how RIPPER and CN2 performed in the complex domain of credit score classification by evaluating a variety of performance metrics such as accuracy, precision, recall, and F1 score. This section contains brief explanations for the research question(RQ). The content briefly detailed the response to the RQ, providing insights into the study's important conclusions.

RQ. Which algorithm, RIPPER or CN2, is more effective based on each evaluation metric like accuracy, precision, recall, and F1 score, in the context of personal loan credit scoring?
The motivation behind this research question comes from the necessity to determine the most effective algorithm, RIPPER or CN2, for personal loan credit scoring. Given the important necessity of effectively assessing creditworthiness in the financial sector, it is essential to discover which algorithm outperforms other evaluation metrics such as accuracy, precision, recall, and F1 score. By addressing this research question, the study intends to provide useful insights into the relative importance of RIPPER and CN2 in the context of personal loan credit scoring.

Comparison based on the accuracy
The emphasis on accuracy as the major parameter for evaluating the RIPPER and CN2 algorithms throughout the study emphasizes the necessity of correctly classifying cases. Accuracy provides a comprehensive perspective of the model's performance by accounting for both true positives (properly predicted positive occurrences) and true negatives (correctly predicted negative instances) about the total number of instances. When comparing the accuracy of the CN2 and RIPPER algorithms, it is clear that the CN2 algorithm outperforms RIPPER, with a better accuracy rate of 47.51% compared to RIPPER's 38.24%. This difference suggests that the CN2 algorithm had a better capacity to correctly identify cases in both positive and negative categories. The graph shows the comparison of accuracy in Figure 6.1.

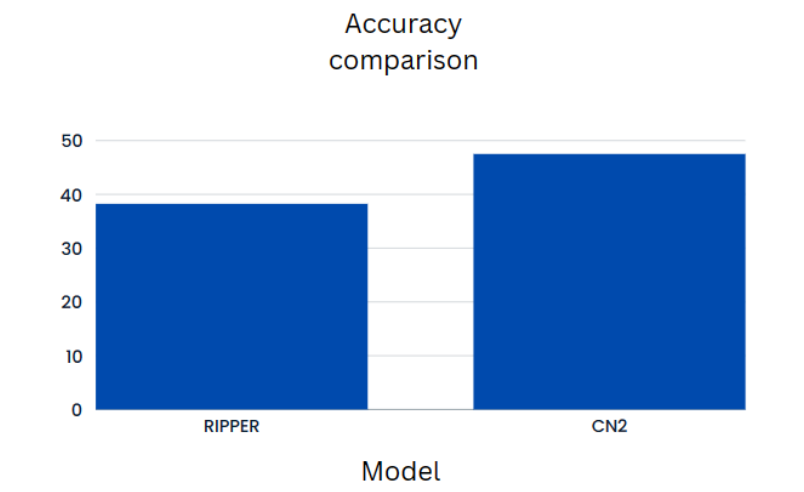


Figure 6.1: Comparison based on accuracy

Comparison based on the Precision

Precision was used as an additional parameter to evaluate the effectiveness of the algorithms. To determine which algorithm performed better, the thesis compared its precision to that of the testing dataset. Following investigation, a slight discrepancy between the two algorithms was discovered. The CN2 algorithm had a greater precision (42%) than the RIPPER approach, which had a slightly lower precision of 34%. The higher precision of the CN2 algorithm suggests that it can reduce false positives more effectively than RIPPER. This suggests that the CN2 algorithm may have been more effective at precisely recognizing positive instances while maintaining a low rate of error. The graph shows the comparison of precision in Figure 6.2.

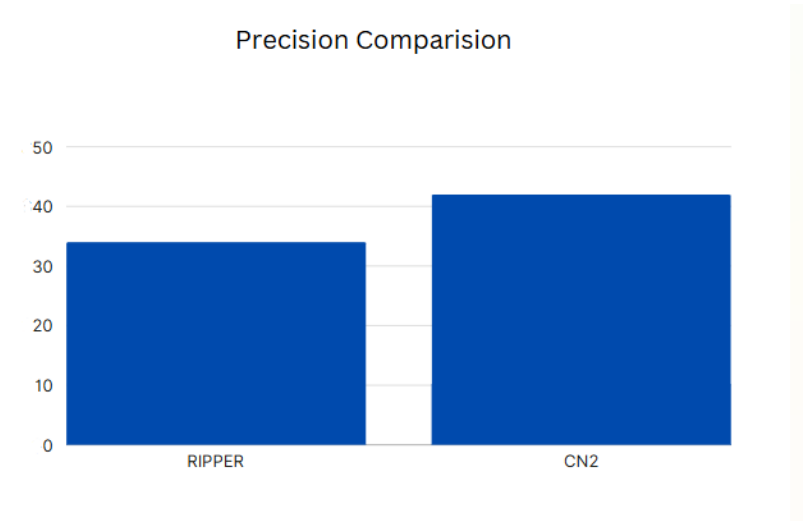


Figure 6.2: Comparison based on precision

Comparison based on the Recall

Recall was used as an evaluation parameter to determine the most successful algorithm among those chosen. The testing dataset was used to evaluate the recall of the selected algorithms, allowing for a comparison of their performance. It was discovered that the RIPPER algorithm had a higher recall of 58%, followed by CN2 at 43%. Examining recall alongside true positives and negatives yields extra information about the algorithms' performance. Recall, also known as sensitivity, is the proportion of real positive instances that the model correctly recognized out of all actual positive instances. A higher recall shows that the model is better at capturing positive instances, which is critical for applications where recognizing all positive cases is a top priority, even if it means generating more false positives. The graph shows the comparison of recall in Figure 6.3.

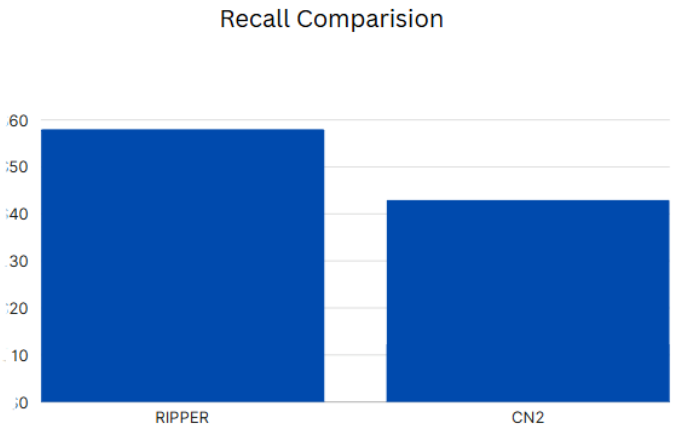


Figure 6.3: Comparison based on recall

Comparison based on the F1 score
The F1 score was used as an additional performance parameter to assess the success of the algorithms. After analyzing the results, it was discovered that the CN2 algorithm had a higher F1 score of 42%, followed by RIPPER with a score of 38%. The F1 score, when combined with actual positives and negatives, provides additional insight into the algorithms' performance. The F1 score is the harmonic mean of precision and recall, and it provides a balanced evaluation of a model's performance for both positive and negative classifications. In this context, the greater F1 score of the CN2 algorithm indicates that it achieved a better balance of precision and recall than RIPPER. This suggests that the CN2 algorithm may have performed better in reliably classifying both positive and negative cases while minimizing false positives and negatives. The graph shows the comparison of recall in Figure 6.4.

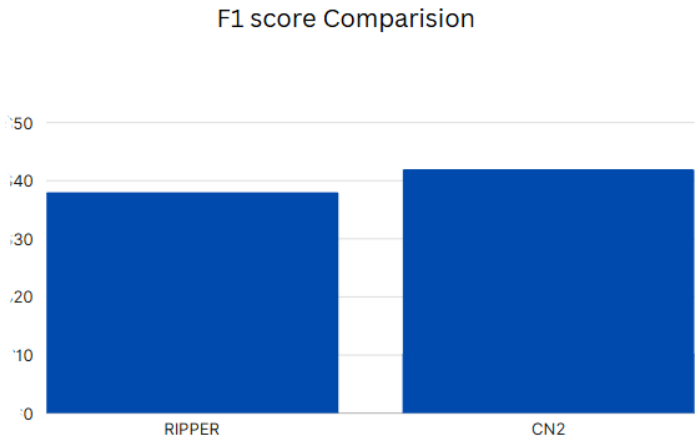


Figure 6.4: Comparison based on F1 score

6.1 Reflections

Several performance metrics were thoroughly examined to determine the usefulness of the RIPPER and CN2 algorithms for personal loan scoring. Each metric provided distinct insights into the algorithms' capabilities and flaws, allowing for a more comprehensive understanding of their performance. Accuracy, the major indicator, provided a broad overview of the algorithms' overall predictive power. CN2 emerged as the top performer, with better accuracy than RIPPER. This means that CN2 is more qualified to generate accurate predictions about loan applicants' creditworthiness, which will lead to more trustworthy lending decision-making procedures. Precision, which assesses the algorithms' ability to properly identify positive cases out of all cases predicted as positive, indicated significant disparities between the two models. CN2 had greater precision, suggesting its ability to minimize false positive predictions. RIPPER, on the other hand, had slightly worse precision, indicating a

larger risk of misclassifying events as positive when they are negative.

Recall, a parameter that measures the algorithms' ability to catch all relevant instances, revealed differences in performance between RIPPER and CN2. While RIPPER had a greater recall, demonstrating its effectiveness in detecting positive events, CN2 was not far behind. This implies that CN2 may neglect some positive cases while compensating by making more accurate predictions overall. The F1 score, which balances precision and recall, offered useful information about the algorithms' overall performance. CN2 had a higher F1 score than RIPPER, indicating a better balance between accurately recognizing positive cases and reducing false positives. This shows that CN2 strikes a better balance between precision and recall, resulting in more robust prediction skills in the context of personal loan credit scoring. Overall, a complete study of these metrics demonstrates CN2's advantage over RIPPER in personal loan scoring. While RIPPER excels in some areas, such as recall, CN2 provides more balanced and reliable performance across various parameters, making it the best option for predictive modeling in this domain.

In a thorough comparison of the RIPPER and CN2 algorithms for personal loan scoring, CN2 emerged as the more effective algorithm overall. Through a comprehensive analysis of many performance metrics, including accuracy, precision, recall, and F1 score, CN2 consistently beat RIPPER across various predictive modeling dimensions. CN2 outperformed RIPPER in accuracy, precision, and F1 score, showing that it can make accurate predictions while minimizing false positives when identifying creditworthy loan applicants. While RIPPER had a somewhat higher recall, indicating its effectiveness in catching positive cases, CN2 compensated by striking a better balance between precision and recall. CN2's more balanced and consistent performance across all measures makes it a superior option for personal loan credit scoring applications.

Chapter 7

Conclusions and Future Work

7.1 Conclusion

Credit scoring is a mechanism used by lenders to determine the creditworthiness of applicants for loans or credit. A credit score is calculated by collecting and evaluating data such as financial history, income, and debts. This score allows lenders to assess the risk of lending to an application and make informed decisions about loan acceptance, terms, and interest rates. Finally, credit scoring allows lenders to properly manage risk and extend credit to borrowers who are more likely to repay their loans responsibly.

The thesis used a credit score classification dataset from Kaggle and evaluated multiple metrics, including accuracy, precision, recall, and F1 score. CN2 outperformed the two selected machine learning algorithms, RIPPER and CN2. CN2 performs better in accuracy, precision, and f1 score and RIPPER performs better in recall. The findings showed that CN2 surpassed RIPPER in all metrics except recall. CN2's better accuracy means that it made more correct classifications than RIPPER over the entire dataset. Precision favored CN2. A higher precision indicates that CN2 had fewer false positives than RIPPER, implying a greater capacity to reliably identify positive instances. However, in terms of recall, RIPPER performed better. Despite CN2's general superiority in accuracy and precision, its less favorable recall indicates that it may have overlooked some favorable cases when compared to RIPPER. The F1 score, which considers both precision and recall, favors CN2. This suggests that CN2 achieved a more optimal trade-off between precision and recall compared to RIPPER.

The interpretability of algorithm-generated credit scoring rules is essential for facilitating loan decision-making processes. When rules are understandable to users, such as loan officers or borrowers, they can provide straightforward explanations for why a certain lending decision was reached. This transparency not only fosters trust but also enables more informed decisions. For example, if a rule indicates that people with a credit score over one level and a debt-to-income ratio below another are eligible for a loan, both loan officers and borrowers can comprehend the criteria used to determine creditworthiness. This transparency allows borrowers to take steps to enhance their credit history as needed, and loan officers to properly communicate lending choices. In conclusion, the thesis determined that CN2 outperforms RIPPER in the domain of personal loan credit scoring making it the better choice for

predictive modeling in this domain.

7.2 Future Work

1. To examine other techniques, compare RIPPER and CN2 to different credit scoring machine learning algorithms.

2. Improve the explainability and interpretability of RIPPER and CN2 models by techniques such as rule extraction and visualization.

3. Investigate ensemble approaches like bagging, boosting, or stacking to combine the strengths of RIPPER and CN2 for better prediction accuracy and robustness.

References

[1] "Applied Sciences | Free Full-Text | Semantic Description of Explainable Machine Learning Workflows for Improving Trust." [Online]. Available: https://www.mdpi.com/2076-3417/11/22/10804

[2] "Credit Score Classification." [Online]. Available: https://www.kaggle.com/datasets/mohammedobeidat/credit-score-classification

[3] H. A. Abdou and J. Pointon, "Credit Scoring, Statistical Techniques and Evaluation Criteria: A Review of the Literature," *Intelligent Systems in Accounting, Finance and Management*, vol. 18, no. 2-3, pp. 59–88, 2011, _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/isaf.325. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/isaf.325

[4] C. C. Aggarwal, "Data Classification," in *Data Mining: The Textbook*, C. C. Aggarwal, Ed. Cham: Springer International Publishing, 2015, pp. 285–344. [Online]. Available: https://doi.org/10.1007/978-3-319-14142-8_10

[5] S. Asadi and J. Shahrabi, "Complexity-based parallel rule induction for multiclass classification," *Information Sciences*, vol. 380, pp. 53–73, Feb. 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0020025516314748

[6] M. Bhattacharya and D. Datta, "Diabetes Prediction using Logistic Regression and Rule Extraction from Decision Tree and Random Forest Classifiers," in *2023 4th International Conference for Emerging Technology (INCET)*, May 2023, pp. 1–7. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10170270

[7] M. D. Bloice and A. Holzinger, "A Tutorial on Machine Learning and Data Science Tools with Python," in *Machine Learning for Health Informatics: State-of-the-Art and Future Challenges*, A. Holzinger, Ed. Cham: Springer International Publishing, 2016, pp. 435–480. [Online]. Available: https://doi.org/10.1007/978-3-319-50478-0_22

[8] C. A. Brunk and M. J. Pazzani, "An Investigation of Noise-Tolerant Relational Concept Learning Algorithms," in *Machine Learning Proceedings 1991*, L. A. Birnbaum and G. C. Collins, Eds. San Francisco (CA): Morgan Kaufmann, Jan. 1991, pp. 389–393. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9781558602007500805

[9] J. Błaszczyński, R. Słowiński, and M. Szeląg, "Sequential covering rule induction algorithm for variable consistency rough set approaches," *Information*

Sciences, vol. 181, no. 5, pp. 987–1002, Mar. 2011. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0020025510005359

[10] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, no. 1, p. 6, Jan. 2020. [Online]. Available: https://doi.org/10.1186/s12864-019-6413-7

[11] M. Chisholm and P. Tadepalli, "Learning Decision Rules by Randomized Iterative Local Search," Aug. 2002.

[12] P. Clark and T. Niblett, "The CN2 Induction Algorithm," *Machine Learning*, vol. 3, no. 4, pp. 261–283, Mar. 1989. [Online]. Available: https://doi.org/10.1023/A:1022641700528

[13] W. W. Cohen, "Fast Effective Rule Induction," in *Machine Learning Proceedings 1995*, A. Prieditis and S. Russell, Eds. San Francisco (CA): Morgan Kaufmann, Jan. 1995, pp. 115–123. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9781558603776500232

[14] K. Dembczyński, W. Kotłowski, and R. Słowiński, "Beyond Sequential Covering – Boosted Decision Rules," in *Advances in Machine Learning I: Dedicated to the Memory of Professor Ryszard S.Michalski*, J. Koronacki, Z. W. Raś, S. T. Wierzchoń, and J. Kacprzyk, Eds. Berlin, Heidelberg: Springer, 2010, pp. 209–225. [Online]. Available: https://doi.org/10.1007/978-3-642-05177-7_10

[15] J. Demsar, B. Zupan, G. Leban, and T. Curk, "Orange: From Experimental Machine Learning to Interactive Data Mining, White Paper," vol. 3202, Jan. 2004, pp. 537–539.

[16] S. K. Dey, A. Hossain, and M. M. Rahman, "Implementation of a Web Application to Predict Diabetes Disease: An Approach Using Machine Learning Algorithm," in *2018 21st International Conference of Computer and Information Technology (ICCIT)*, Dec. 2018, pp. 1–5. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8631968

[17] N. Elmrabit, F. Zhou, F. Li, and H. Zhou, "Evaluation of Machine Learning Algorithms for Anomaly Detection," in *2020 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*, Jun. 2020, pp. 1–8. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9138871

[18] M. Fareez, V. Thangarajah, and S. Saabith, "POPULAR PYTHON LIBRARIES AND THEIR APPLICATION DOMAINS," Nov. 2020.

[19] A. Fernández, S. García, J. Luengo, E. Bernadó-Mansilla, and F. Herrera, "Genetics-Based Machine Learning for Rule Induction: State of the Art, Taxonomy, and Comparative Study," *IEEE Transactions on Evolutionary Computation*, vol. 14, no. 6, pp. 913–941, Dec. 2010, conference Name: IEEE Transactions on Evolutionary Computation. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/5491152?casa_token=qxK9SI7ds1oAAAAA:5ZY7VDoF6gkGJScgTWu4QzkDk2Tvngm6hicczR_GBOYzi2lFOdY0NbrK6fvOPeHoI3OzyLbn1YU

[20] J. Fürnkranz and G. Widmer, "Incremental Reduced Error Pruning," in *Machine Learning Proceedings 1994*, W. W. Cohen and H. Hirsh, Eds. San Francisco (CA): Morgan Kaufmann, Jan. 1994, pp. 70–77. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9781558603356500179

[21] J.-S. Hong, J. Lee, and M. K. Sim, "Concise rule induction algorithm based on one-sided maximum decision tree approach," *Expert Systems with Applications*, vol. 237, p. 121365, Mar. 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417423018675

[22] B. Kavšek and N. Lavrač, "Apriori-Sd: Adapting Association Rule Learning to Subgroup Discovery," *Applied Artificial Intelligence*, vol. 20, no. 7, pp. 543–583, Sep. 2006, publisher: Taylor & Francis _eprint: https://doi.org/10.1080/08839510600779688. [Online]. Available: https://doi.org/10.1080/08839510600779688

[23] P. Lemenkova, "Python Libraries Matplotlib, Seaborn and Pandas for Visualization Geo-spatial Datasets Generated by QGIS," Rochester, NY, Sep. 2020. [Online]. Available: https://papers.ssrn.com/abstract=3699706

[24] C. K. Leong, "Credit Risk Scoring with Bayesian Network Models," *Computational Economics*, vol. 47, no. 3, pp. 423–446, Mar. 2016. [Online]. Available: https://doi.org/10.1007/s10614-015-9505-8

[25] X.-L. Li and Y. Zhong, "An Overview of Personal Credit Scoring: Techniques and Future Work," vol. 2012, Oct. 2012, publisher: Scientific Research Publishing. [Online]. Available: http://www.scirp.org/journal/PaperInformation.aspx?PaperID=24193

[26] W. Liu, H. Fan, and M. Xia, "Credit scoring based on tree-enhanced gradient boosting decision trees," *Expert Systems with Applications*, vol. 189, p. 116034, Mar. 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417421013798

[27] F. Louzada, A. Ara, and G. B. Fernandes, "Classification methods applied to credit scoring: Systematic review and overall comparison," *Surveys in Operations Research and Management Science*, vol. 21, no. 2, pp. 117–134, Dec. 2016. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1876735416300101

[28] R. G. Mantovani, T. Horváth, R. Cerri, J. Vanschoren, and A. C. d. Carvalho, "Hyper-Parameter Tuning of a Decision Tree Induction Algorithm," in *2016 5th Brazilian Conference on Intelligent Systems (BRACIS)*, Oct. 2016, pp. 37–42. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7839559

[29] D. G. Mayer and D. G. Butler, "Statistical validation," *Ecological Modelling*, vol. 68, no. 1, pp. 21–32, Jul. 1993. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0304380093901052

[30] L. Munkhdalai, T. Munkhdalai, O.-E. Namsrai, J. Y. Lee, and K. H. Ryu, "An Empirical Comparison of Machine-Learning Methods on Bank Client Credit Assessments," *Sustainability*, vol. 11, no. 3, p. 699, Jan. 2019, number: 3

Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: https://www.mdpi.com/2071-1050/11/3/699

[31] S. Nandhini and J. Marseline K.S., "Performance Evaluation of Machine Learning Algorithms for Email Spam Detection," in *2020 International Conference on Emerging Trends in Information Technology and Engineering (ic-ETITE)*, Feb. 2020, pp. 1–4. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9077835/references#references

[32] I. Nti, O. Nyarko-Boateng, and J. Aning, "Performance of Machine Learning Algorithms with Different K Values in K-fold Cross-Validation," *International Journal of Information Technology and Computer Science*, vol. 6, pp. 61–71, Dec. 2021.

[33] K. Pimcharee and O. Surinta, "Data Mining Approaches in Personal Loan Approval," vol. 8, pp. 15–21, Jul. 2021.

[34] J. R. Quinlan, "Learning logical definitions from relations," *Machine Learning*, vol. 5, no. 3, pp. 239–266, Aug. 1990. [Online]. Available: https://doi.org/10.1007/BF00117105

[35] R. L. Rivest, "Learning decision lists," *Machine Learning*, vol. 2, no. 3, pp. 229–246, Nov. 1987. [Online]. Available: https://doi.org/10.1007/BF00058680

[36] U. Rückert and L. De Raedt, "An experimental evaluation of simplicity in rule learning," *Artificial Intelligence*, vol. 172, no. 1, pp. 19–28, Jan. 2008. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0004370207001117

[37] J. J. Salazar, L. Garland, J. Ochoa, and M. J. Pyrcz, "Fair train-test split in machine learning: Mitigating spatial autocorrelation for improved prediction accuracy," *Journal of Petroleum Science and Engineering*, vol. 209, p. 109885, Feb. 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0920410521015023

[38] I. Stančin and A. Jović, "An overview and comparison of free Python libraries for data mining and big data analysis," in *2019 42nd International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, May 2019, pp. 977–982, iSSN: 2623-8764. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8757088

[39] L. C. Thomas, "A survey of credit and behavioural scoring: forecasting financial risk of lending to consumers," *International Journal of Forecasting*, vol. 16, no. 2, pp. 149–172, Apr. 2000. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0169207000000340

[40] D. Tripathi, A. K. Shukla, B. R. Reddy, G. S. Bopche, and D. Chandramohan, "Credit Scoring Models Using Ensemble Learning and Classification Approaches: A Comprehensive Survey," *Wireless Personal Communications*, vol. 123, no. 1, pp. 785–812, Mar. 2022. [Online]. Available: https://doi.org/10.1007/s11277-021-09158-9

[41] A. K. H. Tung, "Rule-Based Classification," in *Encyclopedia of Database Systems*, L. Liu and M. T. Özsu, Eds. New York, NY: Springer, 2016, pp. 1–4. [Online]. Available: https://doi.org/10.1007/978-1-4899-7993-3_559-2

[42] G. Varoquaux, L. Buitinck, G. Louppe, O. Grisel, F. Pedregosa, and A. Mueller, "Scikit-learn: Machine Learning Without Learning the Machinery," *GetMobile: Mobile Computing and Communications*, vol. 19, no. 1, pp. 29–33, Jun. 2015. [Online]. Available: https://dl.acm.org/doi/10.1145/2786984.2786995

[43] Z. Vujovic, "Classification Model Evaluation Metrics," *International Journal of Advanced Computer Science and Applications*, vol. Volume 12, pp. 599–606, Jul. 2021.

[44] Y. Wang, Y. Zhang, Y. Lu, and X. Yu, "A Comparative Assessment of Credit Risk Model Based on Machine Learning ——a case study of bank loan data," *Procedia Computer Science*, vol. 174, pp. 141–149, Jan. 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050920315830

[45] M.-T. Wu, "Confusion matrix and minimum cross-entropy metrics based motion recognition system in the classroom," *Scientific Reports*, vol. 12, no. 1, p. 3095, Feb. 2022, publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/s41598-022-07137-z

Appendix A

Supplemental Information



Faculty of Engineering, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden